

Building Blocks of Program Reasoning: Composing Obfuscation Specialists by Merging

ANONYMOUS AUTHOR(S)

Software developers and malware authors widely use code obfuscation to conceal program logic. In real-world scenarios, these actors rarely rely on isolated techniques. Instead, they stack multiple transformations together to build resilient, layered defenses. While large language models (LLMs) can effectively reason about clean source code, their performance drops significantly when they encounter these complex defenses. Despite stacked transformations being the realistic standard in practice, current literature almost exclusively evaluates models against individual, isolated obfuscations. Existing approaches fundamentally fail to assess and effectively handle layered techniques, and learning to counter individual transformations does not automatically help models handle stacked ones. To address this critical gap, we evaluate model merging as a solution for realistic stacked obfuscation. We compare this approach against three fine-tuning baselines, which achieve strong performance on familiar transformations but struggle to generalize beyond their training distribution and can exhibit catastrophic forgetting of untrained reasoning capabilities. Model merging mitigates these limitations and achieves the strongest overall balance between performance on stacked obfuscations and preservation of untrained reasoning capabilities. Merged models significantly outperform their base models on obfuscated code and, surprisingly, also improve prediction on the original, clean programs. They additionally preserve performance on an untrained reverse-reasoning task, where the fine-tuning baselines frequently degrade. By successfully combining specific adaptations with broader reasoning capabilities, these results demonstrate that merging independently learned models is a highly effective strategy for analyzing complex, realistically obfuscated code.

1 Introduction

Code obfuscation is widely used to make software difficult to inspect while preserving its intended functionality. In benign settings, developers employ obfuscation to protect intellectual property, implementation details, and proprietary business logic. In adversarial settings, however, malware authors routinely exploit the same techniques to conceal malicious behavior and frustrate program analysis and reverse engineering. Common transformations include identifier renaming, control-flow restructuring, dead-code insertion, expression rewriting, call indirection, etc [7]. Although such transformations are intended to preserve program semantics, they can substantially obscure how those semantics are expressed in the source code, thereby increasing the difficulty of understanding the behavior of malware code [7, 8, 26, 30].

This difficulty is well established for human analysts. Nguyen *et al.* [22], for example, showed that code obfuscation significantly reduces humans' accuracy in reasoning about program outputs. Such findings are particularly important for malware analysis, where analysts must often infer the behavior of code whose surface structure has been deliberately engineered to resist comprehension.

Large language models (LLMs) offer a promising alternative for automating such reasoning. Models have demonstrated strong capabilities in code generation, code understanding, summarization, and vulnerability detection [4, 9, 14, 19, 23]. However, most evaluations are performed on naturally written programs, where meaningful identifiers, familiar control structures, and common coding idioms provide abundant lexical and structural cues. Obfuscated programs remove or distort many of these cues while retaining the underlying computation, creating a useful stress

test of whether an LLM actually reasons about program behavior or instead depends on surface regularities.

Recent studies suggest that this distinction matters. Rong *et al.* [27] showed that semantics-preserving obfuscations can cause substantial degradation in LLM program reasoning, including program-output prediction. Similarly, Nikiema *et al.* [24] reported a statistically significant decline in model performance as the complexity of obfuscation increases. Together, these findings indicate that this limitation is especially problematic for security applications: an AI-assisted malware analyzer should not cease to understand a malicious computation merely due to obfuscation.

An important challenge, however, has received considerably less attention: *real obfuscated programs rarely contain only one transformation*. Obfuscators can combine multiple techniques so that, for example, renamed identifiers are embedded inside flattened control flow, rewritten expressions, and additional irrelevant code. Such **stacked obfuscation** creates a fundamentally harder deployment setting. Success on each transformation independently does not necessarily imply success when those transformations occur together. Consequently, simply demonstrating that a model can be adapted to individual obfuscations is insufficient. For practical malware analysis, the learned capability must also *compose*.

This leads to our central question: *How should capabilities learned from individual obfuscations be composed so that an LLM can reason robustly about programs containing multiple transformations without sacrificing capabilities that are not reinforced during adaptation?* To address this question, we compare approaches that compose obfuscation-specific knowledge at different stages of the pipeline: breadth training, which exposes a single adapter to multiple transformations; paired-consistency training, which regularizes an obfuscated-code student against a clean-code teacher; inference-time routing, which maintains separate specialists and combines them dynamically; and parameter-space merging, which combines independently trained specialists before inference.

Our evaluation reveals a consistent trade-off between *specialization* and *capability preservation*. Monolithic fine-tuning methods achieve the highest peak accuracy when evaluation closely matches the transformation distribution and reasoning direction used during training. However, this specialization can come at a cost: performance degrades more sharply when the model is evaluated on an unseen obfuscation family or asked to reason in an untrained reverse direction. In particular, fine-tuned models frequently lose capability that was present in the base model even as their performance improves on the trained forward task.

In this paper, we propose composing independently trained obfuscation specialists using **model merging**. Rather than asking a single optimization trajectory to internalize all transformations jointly, merging first learns each transformation independently and then combines the resulting parameter updates into one adapter. The resulting model requires neither an expert bank nor an inference-time router. Although merged models do not always achieve the highest in-distribution accuracy, they remain competitive on familiar stacked obfuscations while preserving substantially more capability outside the trained objective. Across the evaluated regimes, model merging therefore achieves the strongest overall balance between performance on stacked obfuscations and preservation of untrained reasoning capabilities.

This distinction becomes clear when the reasoning task is reversed. Our adapters are trained only for forward program reasoning, yet we also evaluate whether they can infer an input that produces a specified output. Several monolithic fine-tuning approaches degrade below the untuned model on this task, despite improving substantially on forward reasoning. Merged models, by contrast, largely preserve or improve this pre-existing capability. We further find that many fine-tuned failures are not arbitrary: the model often returns the correct *forward* answer to a backward prompt, indicating strong specialization to the reasoning direction reinforced during training.

Merged models also substantially outperform their corresponding base models on the original unobfuscated programs. Because the default merge includes a clean-code specialist in addition to five obfuscation specialists, this result by itself does not identify which specialists are responsible for the clean-code gain. We therefore treat clean-code performance as evidence that merging can preserve general program-reasoning capability while composing obfuscation-specific adaptations, and separately examine specialist subsets to test whether this capability also emerges from the obfuscation specialists alone.

We hypothesize that the robustness of merging arises from the independence of the specialist training trajectories. Semantics-preserving obfuscations provide multiple *views* of the same underlying computation. A specialist may therefore learn both transformation-specific information associated with one surface representation and updates that support reasoning about the computation shared across representations. Training these specialists independently avoids forcing all such signals through a single joint optimization trajectory. Parameter-space composition can then combine complementary updates while reducing the degree to which adaptation is dominated by one transformation or reasoning direction. We test this hypothesis through clean-code evaluation, specialist-subset ablations, unseen transformations, stacked obfuscations, and reversed reasoning.

These results lead to a broader conclusion about robustness to code obfuscation. The challenge is not simply to expose a model to more transformations. Increasing training coverage can improve peak performance on familiar conditions while simultaneously narrowing the set of capabilities preserved by the adapted model. Robust adaptation therefore depends not only on *what* transformations are observed, but also on *how* the resulting knowledge is composed. Model merging provides a way to retain most of the adaptation benefit while reducing this loss of untrained capability.

Contributions. This paper makes the following main contributions:

1. A systematic study of compositional robustness under stacked obfuscation. We study whether capabilities learned from individual obfuscation transformations compose when multiple transformations occur simultaneously, reflecting an important characteristic of real obfuscations.

2. An empirical characterization of specialization and forgetting under obfuscation adaptation. We show that conventional fine-tuning achieves strong performance on familiar transformations and stacked combinations, but can become increasingly specialized to the observed transformation distribution and reasoning direction. This specialization can degrade capabilities that are not reinforced by the training objective, including reverse program reasoning.

3. Model merging as a mechanism for balancing adaptation and capability preservation. Parameter-space merging remains competitive with the strongest fine-tuning approaches on stacked obfuscations while substantially better preserving performance on an untrained reverse-reasoning task. Across evaluation regimes, it provides the strongest overall balance between adaptation performance and preservation of untrained reasoning capabilities, while producing a single deployed adapter with no expert bank or inference-time router.

4. Evidence that independently trained specialists preserve complementary reasoning capabilities. Through clean-code evaluation, specialist-subset comparisons, unseen transformations, and reversed reasoning, we examine whether independently trained specialists retain capabilities shared across semantics-preserving views in addition to transformation-specific adaptations.

2 Background

Code obfuscation. Code obfuscation transforms a program while making its behavior harder to analyze. Obfuscation techniques vary in what they change and what they preserve. A classical taxonomy divides them into four families [7, 8]: *layout* transforms rewrite identifiers and

remove names, comments, and annotations; *control* transforms restructure the flow of computation; *data* transforms change how values are represented or computed; and *preventive* transforms target analysis tools rather than the program's representation. The first three are typically semantics-preserving, whereas preventive techniques can make behavior dependent on the execution environment through mechanisms such as anti-debugging checks, anti-virtualization probes, and self-modifying code. Obfuscation techniques also operate at different levels. Source- and bytecode-level transformations leave an artifact that can be analyzed statically, whereas packers and virtualization-based techniques may reconstruct the protected code only at runtime [? ?]. Even when a transformation is semantics-preserving, preservation generally refers to observable program behavior: it does not require identical timing, memory usage, or exception behavior.

Our study focuses on semantics-preserving, source-level transformations. This restriction gives us a well-defined behavioral ground truth: because the transformation does not change program semantics, the behavior of an obfuscated program can be determined from its clean counterpart. This makes behavioral questions exactly gradable and allows us to test whether a model remains invariant to changes in program representation.

Predicting program behaviors. We evaluate reasoning about obfuscated programs through two complementary behavior-prediction tasks: *output prediction* and *input prediction*. Given a program and an input, output prediction asks what the program returns. Conversely, given a program and a return value, input prediction asks for an input that produces that value. Current code models remain imperfect at both tasks, even for unobfuscated programs [13]. The two directions probe different aspects of program reasoning. Output prediction follows the program's execution from input to result, whereas input prediction requires reasoning in the reverse direction. Evaluating both directions therefore provides a broader test of behavioral understanding than output prediction alone. This paired formulation is also motivated by prior work showing that reasoning performance can depend strongly on the direction represented during training [1, 2, 15, 25].

This distinction is important for our study. If a model is trained to reason about one direction, improvements in that direction need not transfer to the other. We therefore treat the two tasks as separate but related capabilities, allowing us to measure not only whether a model can reason about obfuscated code, but also whether adaptation to one reasoning direction affects the other.

Adapting a model. We use low-rank adaptation (LoRA) [18] to adapt a code model to individual obfuscation transformations. LoRA freezes the base model weights W and learns a low-rank update $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll d$. The resulting update can be stored separately from the base model and applied to the target modules.

This modularity is central to our study. Each LoRA adapter can be trained as a specialist for a particular obfuscation transformation, after which multiple specialists can be combined without changing the underlying base model. We keep the base model, target modules, and LoRA rank fixed across methods, so the primary differences between methods arise from how training data and learned updates are used rather than from model capacity.

3 Merging Obfuscation Specialists

We train low-rank adapters independently on different program representations and compose the resulting specialists into a single adapter before inference. The default merge contains six adapters: one clean-code specialist, L0, and five obfuscation specialists, L1b, L1r, L2, S1, and S2. Each specialist is optimized independently while the base model remains frozen. After training, the specialists are merged once offline and the individual adapters are discarded.

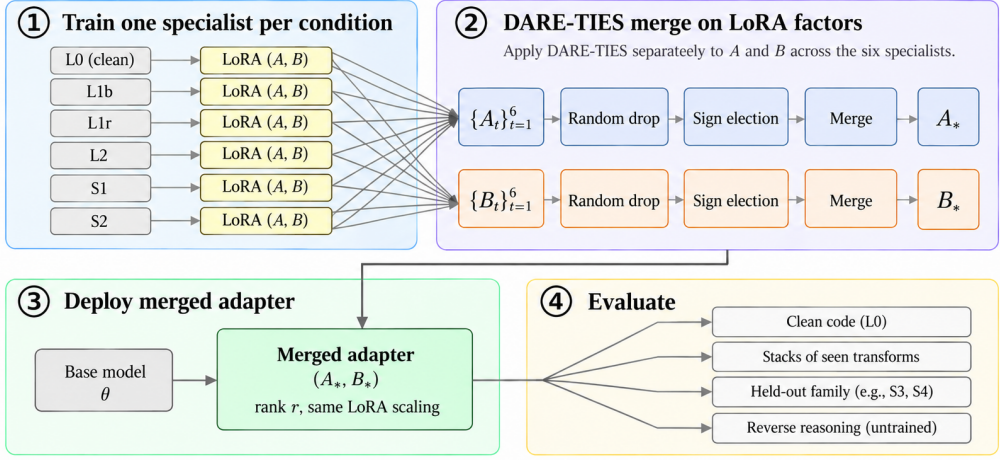


Fig. 1. **Overview of our methodology.** We train six LoRA specialists: one on clean code (L0) and five on seen obfuscations (L1b, L1r, L2, S1, and S2). We merge the corresponding A and B factors separately with DARE-TIES to obtain a single adapter, which is deployed without a router or expert bank and evaluated across clean, stacked, held-out, and reverse-reasoning conditions.

Figure 1 summarizes the complete procedure. Each condition produces an independently trained LoRA specialist; corresponding adapter parameters are then composed offline using DARE-TIES, yielding a single adapter that is deployed without routing or an expert bank.

This separates *specialization* from *composition*. Each adapter learns from a single transformation condition without jointly optimizing against examples from the other conditions. Composition then occurs only after specialist training, producing one adapter with the same inference-time architecture as an ordinary LoRA model. Unlike routing, the deployed model requires neither a gate nor a resident bank of specialists.

Let the frozen base parameters be θ , and let $\Delta\theta_t$ denote the low-rank update learned by specialist t . Independent training produces

$$\{\Delta\theta_1, \dots, \Delta\theta_T\}.$$

We compose these updates using a parameter-space merge operator $\mathcal{M}$,

$$\Delta\theta_{\text{merge}} = \mathcal{M}(\Delta\theta_1, \dots, \Delta\theta_T),$$

and deploy

$$\theta' = \theta + \Delta\theta_{\text{merge}}.$$

LoRA representation. For an adapted weight matrix W , specialist t represents its update as

$$\Delta W_t = B_t A_t,$$

where A_t and B_t are rank- r LoRA factors. All specialists use the same rank, scaling coefficient, and target modules, so corresponding adapter parameters can be merged directly across specialists. The resulting merged adapter has the same rank and target modules as the individual specialists.

Merge operator. We evaluate TIES and DARE-TIES merging. All specialists receive uniform merge weights. TIES first removes low-magnitude parameter changes, determines a consensus sign for each retained coordinate, and averages only updates that agree with that sign. DARE-TIES applies random sparsification and rescaling to the task-specific parameter updates before the TIES sign-resolution and aggregation step. We use a retained density of 0.5 for all models.

The density is fixed across the eight evaluation models. It was selected from $\{0.3, 0.5, 0.7\}$ using a separate 1.5B-parameter pilot model and was not tuned on the reported model panel. Unless otherwise stated, the main results use DARE-TIES; TIES is reported as a merge ablation.

The merge is applied independently to corresponding LoRA parameter tensors at each adapted layer, rather than materializing and merging the full base-model weights.

The operator is not arbitrary. Sparsification and sign election, rather than averaging, do most of the work. Rebuilding the merge with plain uniform averaging at identical ingredients, weights, density, rank and seed (Table 1) does not merely lose: on three of four models it does not clear the format gate on clean code or on the seen single transforms at all, and on the one unseen-family cell where it is readable it reaches 25 % of the untuned model’s clean-code accuracy against an untuned baseline of 57 % – worse than not merging. Averaging is also the only operator that loses the answer contract: it answers the forward question when asked to reverse on 0.174 and 0.428 of items on the two models read in that direction, where TIES sits at 0.000 and DARE-TIES at 0.019 and 0.000. Three operators at one density is not a search, so we do not claim DARE-TIES is optimal, only that the choice is load-bearing.

Training and deployment. Specialists are trained independently using the same optimization setup and base checkpoint. Merging is performed only after specialist training and requires no additional optimization. No transform identity, routing decision, or dynamic expert selection is used during evaluation. Consequently, the deployed artifact is a single low-rank adapter and has the same inference-time parameterization and computational structure as the other single-adapter baselines.

4 Alternative Approaches

We compare specialist merging against six systems that differ in where adaptation occurs: no weight update, prompting, clean-task adaptation, joint training across transforms, consistency-regularized joint training, and inference-time specialist composition. Where applicable, trainable systems use the same base model, LoRA configuration, optimization budget, and underlying training examples.

Untuned. The instruction-tuned base checkpoint evaluated with no parameter updates. This provides the reference performance for each model and condition.

In-context learning. The untuned checkpoint with one solved example prepended to the prompt [?]. The demonstration is a fixed seven-line program written for this study, is not drawn from the evaluation corpus, and is always shown unobfuscated. The same demonstration is used for every model and condition. This baseline measures the effect of providing task-format supervision without parameter adaptation or exposure to obfuscated code.

Clean-code LoRA. A LoRA adapter trained only on unobfuscated programs using the same prompt and optimization schedule as the obfuscation-trained adapters. This baseline isolates task

Table 1. **The merge operator is not arbitrary.** Forward accuracy as a percentage of the untuned model’s clean-code accuracy on the same programs. Ingredients, weights, density, rank and seed are identical across the three merges; only the operator differs. *linear* is plain uniform averaging, with no sparsification and no sign election. **Averaging does not merely lose, it breaks the answer contract:** where it is readable at all it recovers little more than not merging, and it is the only operator with a non-zero direction-lock rate (*lock*: the fraction of backward replies that are the gold *forward* answer). *gated*: every cell of that arm exceeded the 0.25 format-failure gate. *--*: not run — backward and lock were read on two models only, the other two having been cancelled to stay inside a 15-GPU cap once the forward answer was unambiguous on four.

model	operator	L0	singles	unseen	lock
CodeLlama-7B	<i>base</i>	100	75	46	0.000
	LINEAR	<i>gated</i>	<i>gated</i>	<i>gated</i>	0.174
	TIES	142	124	88	0.000
	DARE-TIES	164	147	105	0.019
CodeLlama-13B	<i>base</i>	100	85	57	--
	LINEAR	<i>gated</i>	<i>gated</i>	25	--
	TIES	163	143	95	--
	DARE-TIES	188	168	120	--
Llama-3.1-8B	<i>base</i>	100	85	52	0.000
	LINEAR	115	98	<i>gated</i>	0.428
	TIES	154	133	83	0.000
	DARE-TIES	178	156	104	0.000
Granite-3.1-8B	<i>base</i>	100	92	58	--
	LINEAR	<i>gated</i>	<i>gated</i>	<i>gated</i>	--
	TIES	133	119	72	--
	DARE-TIES	152	132	82	--

adaptation from adaptation to obfuscation. The resulting adapter also serves as the frozen teacher for KL-consistency training.

Pooled training. A single LoRA adapter trained on the same six conditions used by the default merge: L0, L1b, L1r, L2, S1, and S2. The examples are pooled and shuffled during optimization. This provides the data-matched comparison to parameter-space specialist composition.

KL-consistency training. A pooled adapter trained with an additional consistency objective that aligns its predictions on an obfuscated program with those of the frozen clean-code teacher on the corresponding clean parent [?]. For an obfuscated program x_{obf} , clean parent x_{clean} , and gold answer y ,

$$\mathcal{L} = \text{CE}(y \mid x_{\text{obf}}) + \lambda D_{\text{KL}}(p_{\text{teacher}}(\cdot \mid x_{\text{clean}}) \parallel p_{\text{student}}(\cdot \mid x_{\text{obf}})). \quad (1)$$

The KL term is computed over answer tokens only. The teacher is fixed, and the student receives only the obfuscated program as input. Setting $\lambda = 0$ recovers the pooled-training objective.

Router. The transform specialists are kept separate and combined at inference time by a learned gate. At decoder layer input h , the gate assigns weights to expert e using a learned query projection W_q and expert key k_e :

$$a_e = \text{softmax}_e \left(\frac{(W_q \text{LayerNorm}(h))^\top k_e}{\sqrt{d_r} \tau} \right), \quad (2)$$

Table 2. **What each arm costs to deploy.** Adapter parameters shipped alongside the base model, peak allocated memory, and greedy decode throughput on an identical workload (32 prompts, 64 new tokens, one GPU). A single LoRA and the merge ship the same number of parameters. **The router’s cost is storage, not speed:** it holds eight experts resident, 8.1–8.3× the merge’s parameters and 0.4–1.7 GB more peak memory, while its throughput is –8 % to +9 % of a single LoRA’s across the 8 models. Throughput is a single timed run on a shared node and differences of this size are not meaningful. Every adapted arm decodes slower than the untuned model because the LoRA path is not fused here; that cost is shared by all of them.

model	params (M)		peak memory (GB)				throughput (tok/s)			
	LoRA/merge	router	untuned	LoRA	merge	router	untuned	LoRA	merge	router
CodeLlama-7B	80	648	12.6	13.2	13.2	13.8	2490	1120	1115	1131
CodeLlama-13B	125	1015	24.2	25.2	25.2	26.2	1718	884	899	918
CodeLlama-34B	218	1768	62.9	64.5	64.5	66.2	1142	756	756	821
Llama-3.1-8B	84	679	15.0	15.6	15.6	16.3	2313	1057	1054	1072
Granite-3.1-8B	99	802	15.2	16.0	16.0	16.7	893	646	526	654
CodeGemma-7B	100	806	15.9	16.7	16.7	17.4	773	677	677	709
Gemma-3-12B	137	1107	22.7	23.8	23.8	24.8	1017	568	552	568
StarCoder2-15B	48	404	29.7	30.1	30.1	30.5	1868	1256	1206	1154

where d_r is the router dimension and τ is a learned temperature. The base model and specialist adapters remain frozen while the routing parameters are trained. The resulting mixture combines expert updates per token and per layer.

We additionally evaluate three routing controls: **hard routing**, which selects the highest-weight expert; **uniform routing**, which assigns equal weight to all experts; and **random routing**, which freezes the gate at its initialization. These controls separate gains from maintaining a multi-expert mixture from gains attributable to the learned routing function. **Deployment cost.** The router’s cost relative to a merge is a storage cost rather than a speed one, and we measure it rather than assert it (Table 2). On an identical greedy decode workload across all eight models, the router ships 404M–1,768M parameters beside the base model against 48M–218M for the merge (8.1–8.3×) and holds 0.4–1.7 GB more at peak, while its throughput is –8 % to +9 % of a single adapter’s, a spread within the noise of one timed run. Running a gate at every layer and token is therefore not what one pays for; keeping eight experts resident is. Decoding through any adapter costs 12–55 % of the untuned model’s tokens per second in our unfused implementation, but that is shared by every adapted arm and separates none of them.

5 Research Questions

We evaluate three research questions:

- **RQ1: Does specialist merging improve reasoning across clean and obfuscated code?** We evaluate clean code and individual obfuscations to determine whether combining specialists improves performance over the untuned model across both settings.
- **RQ2: Do adaptations learned from individual obfuscations transfer to stacked obfuscation?** We evaluate unseen stacks of increasing depth to determine whether separately learned adaptations remain effective as multiple obfuscations are combined.
- **RQ3: How well do adapted models generalize beyond the training setting?** We evaluate a held-out obfuscation family and reverse reasoning, neither of which is used during adaptation, and whether adaptation preserves general code generation on a standard benchmark.

Table 3. **Dataset summary.** Corpus composition and program-level split.

Corpus		Split and properties	
Dataset	Programs	Property	Value
APPS [?]	1,584	Training programs	1,563
CRUXEval [13]	543	Validation programs	111
HumanEval [3]	104	Held-out programs	557
Total	2,231	Inputs per program	3
		Held-out clean instances	1,671
		Mean program length	8.4 LOC
		Maximum program length	57 LOC
JavaScript	168 programs	JavaScript instances	504

6 Empirical Setup

6.1 Models

We evaluate eight instruction-tuned models spanning four model families: CodeLlama-7B, CodeLlama-13B, and CodeLlama-34B [28]; Llama-3.1-8B [12]; StarCoder2-15B [21]; Gemma-3-12B [10]; CodeGemma-7B [6]; and Granite-3.1-8B [11]. The panel includes both general-purpose and code-specialized models from 7B to 34B parameters. We additionally use the pretrained Llama-3.1-8B checkpoint for the one-shot baseline. All adapted models use rank-32 LoRA with the same target modules.

6.2 Dataset

Our Python corpus contains 2,231 programs drawn from APPS [?], CRUXEval [13], and HumanEval [3]. We retain 1,584 APPS programs, 543 CRUXEval programs, and all 104 HumanEval programs. We split the corpus by program so that no transformed variant of a training program appears in evaluation. Each program is evaluated on three fixed inputs, with reference outputs obtained by executing the clean program.

Coverage varies by transformation because some structural and encoding transforms are not applicable to every program. All paired comparisons are therefore computed over the intersection of programs available in the conditions being compared.

6.3 Obfuscation Types

We organize the obfuscation conditions into **seen single transforms**, **held-out encoding transforms**, and **stacked composites**. See Table 4 for the full definitions and coverage of each condition.

Seen single transforms. The main training set contains five obfuscations: three identifier transforms—L1b, L1r, and L2—and two structural transforms—S1 and S2. L1b performs adversarial identifier renaming, L1r performs random renaming, and L2 minifies identifiers. S1 applies control-flow flattening, while S2 combines opaque predicates with dead-code insertion. Each specialist is trained on exactly one of these conditions, and no specialist is trained on a stacked obfuscation. Together with the clean-code condition L0, these form the six specialist conditions used by the default breadth and merge systems.

Structural components. We additionally separate S2 into its two constituent mechanisms: S3 contains dead-code insertion only, and S4 contains opaque predicates only. These conditions are

Table 4. **Obfuscation conditions.** Single transforms define the atomic conditions used for training and generalization evaluation; stacked conditions are constructed only for evaluation.

Code / group	Family	Transformation	Programs	Role
Clean reference				
L0	none	Normalized source with comments and docstrings removed	557	clean
Seen single-transform conditions				
L1b	identifier	Adversarial renaming of bindings and the entry function	553	train / eval
L1r	identifier	Random hexadecimal renaming of all bindings	557	train / eval
L2	identifier	Sequential identifier minification and removal of type annotations	557	train / eval
S1	structural	Control-flow flattening into a dispatch loop with randomized states	416	train / eval
S2	structural	Opaque predicates combined with never-called dead helper functions	556	train / eval
Structural components used in stacks				
S3	structural	Dead-code insertion using never-called helper functions	557	stacks
S4	structural	Opaque predicates inserted at statement boundaries	556	stacks
Held-out encoding family				
X1	encoding	String encoding and semantics-preserving arithmetic rewriting	405	held-out eval
X1m	encoding	Arithmetic-rewriting component of X1	351	held-out eval
X1s	encoding	String-encoding component of X1	246	held-out eval
H1	encoding	Base64/RC4 string encoding with mixed-boolean-arithmetic rewriting	–	quarantined
Stacked evaluation conditions				
Seen depth-2	composite	Two sequentially applied seen transforms	varies	6 conditions
Seen depth-3/4	composite	Three or four sequentially applied seen transforms	varies	4 conditions
Held-out stacks	composite	Stacks containing X1, X1m, or X1s together with seen transforms	varies	6 conditions

used to construct and analyze stacked composites; they are not part of the five-transform training set used by the breadth and merge systems.

Held-out encoding family. The encoding family is excluded from the main systems’ training data. X1 combines string encoding with semantics-preserving arithmetic rewriting. X1m contains only the arithmetic component, and X1s contains only the string-encoding component. H1 is a quarantined held-out obfuscator using a distinct Base64/RC4-based string encoding together with mixed-boolean-arithmetic rewriting. Thus, X1, X1m, and X1s define the held-out encoding conditions used in the main generalization experiments, while H1 is reserved as the quarantined reference family.

Stacked composites. Stacked conditions apply multiple transforms sequentially to the same clean program, with order treated as part of the condition because later transforms may modify code introduced by earlier ones. We evaluate sixteen stacks: six depth-2 stacks composed only of seen transforms (L1b, L1r, L2, S1, S2, S3, or S4); four deeper stacks of seen transforms at depth 3 or 4; and six stacks that contain the held-out encoding family through X1, X1m, or X1s. No system is trained directly on any stacked condition.

6.4 Systems

All trainable systems use rank-32 LoRA adapters with $\alpha = 64$ and identical target modules. Systems differ only in their training data or composition mechanism as described in Sections 3 and 4.

Table 5. **The seed noise floor.** Range (max – min) in accuracy points across three independently seeded runs of the same recipe — seeds 17, 101 and 202 — on CodeLlama-7B. **The noise is not uniform:** it is roughly three times wider on the held-out family than on clean code, and that is the condition the paper’s sharpest comparisons are made on. These are *training* variation and are disjoint from the program-clustered bootstrap intervals reported elsewhere, which measure sampling variation over programs and contain no seed effect at all. A narrated difference smaller than the relevant entry is not distinguishable from a re-rolled seed. Measured on one model; other models are not measured and may differ.

arm	L0	singles	unseen (X1)
L0	0.60	0.66	0.58
L1b	1.44	0.28	2.39
L1r	0.24	0.90	1.57
L2	0.66	0.53	2.31
S1	0.06	0.63	0.74
S2	1.32	0.30	1.65
<i>breadth</i>	1.50	1.44	0.08
median	0.66	0.63	1.57

6.5 Measurement

Let p_i denote a program, x_i an input, and $y_i = p_i(x_i)$ its reference output.

Seed noise floor. A bootstrap over programs does not contain training variation, so we measure that separately. Three independently seeded runs of the same recipe — seeds 17, 101 and 202, six specialists and pooled training, on CodeLlama-7B — give a median range of 0.66 accuracy points across 21 arm $\times$ condition-group combinations, with a maximum of 2.39 (Table 5). The noise is not uniform: it is roughly three times wider on the held-out family (median 1.57) than on clean code (median 0.66). The two quantities answer different questions and neither subsumes the other, so we do not narrate single-seed differences below roughly one point on clean code or the seen singles, or below roughly 2.4 points on the held-out family. Measured on one model; the other seven are not measured and may differ.

Forward accuracy. Given (p_i, x_i) , the model predicts an output $\hat{y}_i$. We measure normalized exact-match accuracy:

$$\text{Acc}_{\text{fwd}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{norm}(\hat{y}_i) = \text{norm}(y_i)].$$

Backward accuracy. Given (p_i, y_i) , the model predicts an input $\hat{x}_i$. Because multiple inputs may produce the same output, correctness is determined by execution:

$$\text{Acc}_{\text{bwd}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[p_i(\hat{x}_i) = y_i].$$

We additionally report **reference-input recovery**, the fraction of predictions that exactly recover the reference input x_i .

Statistical comparisons. All system comparisons are paired over the intersection of programs available in the compared conditions. We report confidence intervals from a program-clustered bootstrap with 2,000 resamples.

Table 6. **Performance gains across clean and individually obfuscated code.** Forward exact-match accuracy on clean code (L0) and mean accuracy over the five individual seen obfuscations (L1b, L1r, L2, S1, and S2). Green and red denote performance above and below a readable untuned baseline, respectively. Bold denotes the strongest adapted method in each row. “–” indicates that no readable untuned baseline is available.

Model	Condition	Base	Clean	Breadth	KL	Router	Merge
CodeLlama-7B	Clean	.257	.427	.413	.422	.429	.426
	Singles	.194	.377	.389	.402	.393	.380
CodeLlama-13B	Clean	.252	.469	.446	.462	.477	.477
	Singles	.215	.410	.423	.442	.433	.424
CodeLlama-34B	Clean	–	.520	.494	.525	.526	.527
	Singles	.214	.462	.469	.496	.484	.460
Llama-3.1-8B	Clean	.258	.448	.426	.445	.475	.462
	Singles	.219	.390	.393	.416	.427	.406
StarCoder2-15B	Clean	–	.542	.529	.532	.546	.539
	Singles	–	.482	.492	.512	.503	.475
Gemma-3-12B	Clean	–	.520	.507	.502	.530	.523
	Singles	–	.462	.474	.485	.492	.479
CodeGemma-7B	Clean	.237	.465	.439	.445	.474	.465
	Singles	.200	.410	.422	.429	.449	.414
Granite-3.1-8B	Clean	.285	.430	.416	.396	.438	.433
	Singles	.262	.363	.391	.378	.402	.377

7 Results

We organize the results around three questions. First, does specialist merging improve reasoning on clean and individually obfuscated code? Second, do the adaptations learned from individual obfuscations still work when those obfuscations are stacked? Third, how do the methods behave outside the training setting, on an unseen obfuscation family and on reverse reasoning?

7.1 RQ1: Does Specialist Merging Improve Program Reasoning Across Clean and Obfuscated Code?

Table 6 compares performance on clean code and on the individual obfuscations used to train the specialists.

Merging improves clean-code performance. On L0, the merged adapter substantially exceeds the untuned model wherever a readable baseline is available, by 14.8–22.8 percentage points. It is statistically indistinguishable from direct clean-code tuning on all eight models: the paired differences range from –0.3 to +1.4 points and every 95% interval contains zero. Combining several specialists therefore does not require giving up the gain obtained from clean-code training.

Because the default merge contains the L0 specialist, the clean-code gain could come from that specialist alone. Table 7 rules this out on four models. A merge of the five obfuscation specialists without L0 reaches the same clean-code accuracy as the six-way merge; every paired interval for the difference contains zero. A merge of only the two structural specialists does as well. The forward gain is therefore shared by the obfuscation specialists rather than carried by the clean one.

Table 7. **Merge ingredient ablation.** Accuracy as a percentage of the untuned model’s clean-code accuracy on the same programs, on clean code (L0) and pooled over the three seen depth-3 stacks. All four merges use DARE-TIES at identical density, weights and rank and differ only in their specialists: *struct* S1+S2, *ident* L1b+L1r+L2, *no-L0* the five obfuscation specialists, *6-way* the default merge. Forward on L0, every *no-L0-6-way* interval contains zero. --: every cell of that arm exceeded the 0.25 format gate.

model	test	forward					backward				
		base	struct	ident	no-L0	6-way	base	struct	ident	no-L0	6-way
CodeLlama-7B	L0	100	166	166	164	166	100	100	111	115	115
	depth 3	53	140	129	132	129	105	--	98	114	114
CodeLlama-13B	L0	100	191	188	190	189	100	--	--	107	119
	depth 3	63	154	150	151	150	108	--	115	122	125
Llama-3.1-8B	L0	100	175	180	180	178	100	76	90	97	101
	depth 3	66	145	136	140	137	77	67	79	88	91
Granite-3.1-8B	L0	100	149	147	153	152	100	76	76	99	105
	depth 3	85	125	112	118	115	109	68	70	90	95

The improvement extends to individual obfuscations. The same merged adapter is also substantially stronger than the untuned model on the individual identifier and structural obfuscations wherever a readable baseline is available. KL-consistency and routing often achieve higher peak accuracy on these individual conditions, but the merge combines improvements across clean and obfuscated code within a single adapter.

Takeaway. Specialist merging produces large gains over the untuned model on both clean and individually obfuscated code. On clean code, it is indistinguishable from clean-only tuning on all eight models, showing that multiple specialized adaptations can be combined without giving up the gains of direct clean-code training.

7.2 RQ2: Do Separately Learned Adaptations Work When Obfuscations Are Stacked?

Table 8 evaluates the main compositional setting of the paper. Each system is trained only on individual transformations, while evaluation applies two, three, or four obfuscations to the same program. No system sees these exact stacks during training. The table also includes each untuned model’s performance on clean L0 code as a reference.

Training on individual obfuscations transfers to unseen stacks. Across all eight models, adaptation remains effective when transformations are combined. Wherever a readable untuned baseline is available, every adapted system remains substantially stronger than the base model even at depth 4. The gain therefore does not depend on seeing the exact stack during training.

Adapted models can outperform the original model even on clean code. The clean-code reference in Table 8 provides a stronger comparison than the stack baseline alone. Although additional obfuscations make the task progressively harder for the untuned model, adaptation is strong enough to more than recover that loss.

At depth 4, the strongest adapted method exceeds the original model’s L0 accuracy on every model for which that clean baseline is readable, by 4.9–12.4 percentage points. The merge itself exceeds the clean-code baseline on four of those five models: by 3.3 points on CodeLlama-7B, 6.8

Table 8. **Stacked-obfuscation performance relative to clean code.** Forward exact-match accuracy by stack depth. **Base L0** is the untuned model on clean code; Δ vs. L0 is **Merge** – **Base L0** in percentage points. Positive values mean that the merged model on obfuscated code outperforms the untuned model on clean code.

Model	Base L0	Depth	Stack base	Clean	Breadth	KL	Router	Merge	Δ vs. L0 (pp)
CodeLlama-7B	.257	2	.157	.327	.365	.378	.363	.339	+8.2
		3	.135	.328	.366	.373	.359	.328	+7.1
		4	.089	.278	.336	.356	.334	.290	+3.3
CodeLlama-13B	.252	2	.185	.362	.394	.407	.404	.381	+12.9
		3	.159	.354	.395	.406	.399	.375	+12.3
		4	.117	.300	.353	.360	.363	.320	+6.8
CodeLlama-34B	–	2	.180	.401	.432	.459	.437	.408	–
		3	.141	.394	.432	.452	.436	.407	–
		4	.105	.316	.372	.413	.380	.351	–
Llama-3.1-8B	.258	2	.182	.332	.368	.388	.394	.358	+10.0
		3	.168	.322	.364	.380	.389	.355	+9.7
		4	.140	.263	.335	.358	.342	.310	+5.2
StarCoder2-15B	–	2	–	.428	.452	.480	.459	.422	–
		3	–	.431	.455	.478	.456	.425	–
		4	–	.357	.412	.437	.401	.358	–
Gemma-3-12B	–	2	–	.395	.436	.448	.440	.409	–
		3	–	.388	.432	.449	.444	.410	–
		4	–	.302	.394	.413	.383	.332	–
CodeGemma-7B	.237	2	.164	.354	.396	.395	.403	.358	+12.1
		3	.147	.344	.390	.394	.408	.351	+11.4
		4	.124	.271	.361	.359	.360	.303	+6.6
Granite-3.1-8B	.285	2	.236	.314	.364	.356	.363	.330	+4.5
		3	.240	.309	.362	.354	.361	.326	+4.1
		4	.203	.258	.325	.334	.331	.281	–0.4

on CodeLlama-13B, 5.2 on Llama-3.1-8B, and 6.6 on CodeGemma-7B. On Granite-3.1-8B, it is only 0.4 points below the clean baseline.

For example, CodeLlama-7B falls from .257 on clean code to .089 when four obfuscations are stacked. After adaptation, however, all methods score between .278 and .356 on that same depth-4 condition. In other words, an adapted model reasoning over heavily obfuscated code can outperform the original model reasoning over unobfuscated code.

Fine-tuning and routing achieve the highest peak accuracy. KL-consistency is strongest in most model–depth comparisons, while routing and breadth lead the remaining cases. Merging does not maximize forward accuracy in this setting, but it remains far above the untuned stack baseline and usually above the untuned clean-code baseline as well.

Takeaway. Training on individual obfuscations transfers strongly to unseen stacks. Even at depth 4, the strongest adapted system outperforms the untuned model on clean code for every model with a readable L0 baseline. The merged adapter itself does so on four of five such models and is within 0.4 points on the fifth, despite never training directly on stacked examples.

7.3 RQ3: How Well Do the Methods Hold Up Outside the Training Setting?

Table 9 considers two harder cases: an obfuscation family that is never used for training and a reverse-reasoning task that none of the methods are fine-tuned to perform.

Routing performs best on the unseen obfuscation family. Routing achieves the highest forward accuracy on six models, while KL-consistency is strongest on the remaining two. The router is

Table 9. **Performance beyond the training setting.** Forward exact-match accuracy over the held-out encoding-family block and execution-graded accuracy on the reverse-reasoning task. Green and red denote performance above and below a readable untuned baseline, respectively. Bold denotes the strongest adapted method in each row.

Model	Test	Base	Clean	Breadth	KL	Router	Merge
CodeLlama-7B	Held-out	.120	.259	.243	.280	.272	.266
	Reverse	.282	.266	.249	.287	.287	.311
CodeLlama-13B	Held-out	.145	.281	.271	.297	.305	.295
	Reverse	.294	.312	–	.223	.294	.337
CodeLlama-34B	Held-out	.139	.307	.302	.337	.330	.314
	Reverse	.319	.295	–	.260	–	.381
Llama-3.1-8B	Held-out	.135	.243	.233	.260	.279	.263
	Reverse	.322	.282	.264	.273	.296	.361
StarCoder2-15B	Held-out	–	.339	.332	.349	.358	.344
	Reverse	.361	.335	.295	.336	.350	.376
Gemma-3-12B	Held-out	.170	.290	.276	.296	.309	.295
	Reverse	.372	.345	.385	.355	.406	.403
CodeGemma-7B	Held-out	.149	.266	.271	.262	.294	.267
	Reverse	.218	.303	.284	–	.284	.287
Granite-3.1-8B	Held-out	.168	.218	.214	.204	.256	.234
	Reverse	.285	.202	.217	.150	.247	.270

above the merge on all eight models in this block, by 0.8–2.8 points, and the paired interval excludes zero on seven. These margins are of the same size as the seed-to-seed variation on this family (Table 5: median 1.6, maximum 2.4 points), so we claim only that the router is not below the merge, not that it ranks above it. Merging does not provide the highest forward accuracy when the transformation itself is new.

Breadth and merge provide a more direct comparison because they use the same six training conditions. Across the full held-out block, merging exceeds breadth on seven of eight models, significantly on five. The router, by contrast, keeps a larger bank of specialists and chooses among them at inference time, whereas the merge combines its specialists beforehand and deploys only one adapter.

Standard fine-tuning degrades reverse reasoning. The reverse task asks the model to find an input that produces a given output. None of the adaptation methods are trained on this task.

Here, the pattern is almost the opposite of the forward results. Clean-code, breadth, and KL fine-tuning frequently reduce reverse accuracy below the untuned model. Merging instead improves over the untuned model on seven of eight models by 3.0–6.4 percentage points and is the strongest adapted method on six of eight models.

These results suggest that the SFT-based methods often learn the trained input–output mapping too narrowly. Their strong forward performance does not consistently transfer when the same program must be reasoned about in the opposite direction. They often return the program’s *forward output* even though the prompt asks for an input that would produce a given value. Table 10 measures this as the fraction of reverse replies that are exactly the gold forward answer. The untuned model is at 0.000 on seven of eight models and the merge at 0.000–0.001 on all eight, while single

Table 10. **Direction locking.** Fraction of *backward* replies that are exactly the gold *forward* answer: the model was asked for an input and returned the output. Pooled over 16 backward conditions. The untuned models, the in-context (ICL) arm and the merge sit at the floor; every single-adaptor arm is above it. --: arm not run on that model.

model	Base	ICL	Clean	Breadth	KL	Router	Merge
CodeLlama-7B	0.000	0.000	0.081	0.024	0.003	0.015	0.001
CodeLlama-13B	0.000	0.007	0.017	0.047	0.085	0.042	0.001
CodeLlama-34B	0.000	0.000	0.004	0.027	0.010	0.041	0.001
Llama-3.1-8B	0.000	0.000	0.001	0.021	0.024	0.001	0.000
StarCoder2-15B	0.000	0.000	0.152	0.252	0.127	0.133	0.000
Gemma-3-12B	0.000	0.000	0.010	0.002	0.095	0.001	0.000
CodeGemma-7B	0.000	0.000	0.005	0.056	0.668	0.008	0.001
Granite-3.1-8B	0.056	0.050	0.181	0.078	0.296	0.091	0.001

adapters reach 0.181 (Clean), 0.252 (Breadth), 0.296 and 0.668 (KL). This is consistent with memorization of the forward prediction pattern rather than a more general relation between program inputs and outputs.

Prompting does not substitute for this. An in-context baseline, the untuned model given four obfuscated demonstrations in place of the single clean one, is below the one-shot untuned model on six of eight models and below the merge on seven. It does not fall back to the forward answer (at most 0.007), so its shortfall is a smaller gain rather than the failure mode of the tuned arms. Appendix Table 21 separates reverse accuracy into execution-graded accuracy, exact reference-input recovery, format success and direction locking for every model and method.

What the merge retains in reverse depends on its breadth (Table 7). Cutting the merge from five obfuscation specialists to the two structural ones costs 4–9 points of reverse accuracy on clean code wherever the smaller merge is readable. At depth three the six-way merge stays 24–27 points of relative accuracy above the structural merge on Llama-3.1-8B and Granite-3.1-8B, and the structural merge fails the format gate on both CodeLlama models. Forward, the same structural merge is at or above the six-way at depth three on all four models. Breadth buys reverse reasoning, not forward accuracy.

Merging behaves differently. The specialists are optimized independently and combined only after training, rather than being jointly optimized toward the same forward objective. The merged model gives up some peak forward accuracy, but retains substantially more ability to answer a different question about the same program.

Reverse accuracy is graded by execution because more than one input may produce the requested output. Any generated input that returns the correct value is counted as correct. Approximately 15% of items have a trivial target value that a well-formed guess can recover, so Appendix A also reports exact reference-input recovery.

Merging preserves general code generation; single adapters often do not. All adapters are trained only to predict outputs. Table 11 asks whether that costs the ability to write code, on MBPP+ [20] (399 tasks, none in our training data, graded by its extended tests). A TIES merge of the same specialists is never below the untuned model, at +0.7 to +3.8 points on all eight models. The paper’s DARE-TIES merge is at or above the untuned model on four models and 2.3–10.3 points below on the other four. Breadth and KL fall below the untuned model on seven of eight models, and the largest losses are not wrong code but no code: on StarCoder2-15B every single adapter and the router return no function on 98–99% of tasks, typically a predicted value instead. HumanEval+

Table 11. **General coding ability after adaptation.** MBPP+ pass@1 (% , the extended test suite), greedy decoding, 399 tasks, none of which occur in our training data. Green and red denote more than 1.5 points above and below the untuned model; smaller differences are within the run-to-run variation of the untuned model itself. [†]: at least half of the replies contain no function, typically because the model answers with a predicted value. *Merge* is the paper’s DARE-TIES merge; *TIES* is the same specialists merged without random dropping.

Model	Base	ICL	Clean	Breadth	KL	Router	Merge	TIES
CodeLlama-7B	44.1	45.6	26.6	19.8 [†]	2.5 [†]	34.3	41.3	45.4
CodeLlama-13B	48.1	55.6	47.6	33.6	38.4	49.6	50.9	51.9
CodeLlama-34B	52.1	57.4	54.4	34.6	48.1	42.9	46.1	54.4
Llama-3.1-8B	59.2	58.9	58.4	55.9	53.9	59.2	59.7	59.9
StarCoder2-15B	61.7	59.9	0.5 [†]	0.0 [†]	0.5 [†]	0.5 [†]	51.4	62.7
Gemma-3-12B	69.7	71.9	70.4	70.2	68.7	69.7	70.2	71.4
CodeGemma-7B	56.4	52.9	57.4	44.6	16.5 [†]	55.6	54.1	57.4
Granite-3.1-8B	57.9	61.7	60.2	54.4	22.6 [†]	60.2	61.4	60.7

shows the same ordering, but 74 of its 164 problems occur in our training corpus, so we do not rely on it.

These collapses depend on the training seed. Repeating the evaluation with seed-42 adapters on three models, the merges change by at most 4.5 points relative to the untuned model, while single adapters move by up to 56: Breadth on Granite-3.1-8B loses 3.5 points at one seed and 59.9 at the other, and Clean on CodeLlama-7B loses 17.5 and 1.8. A single seed can show that adapters sometimes stop generating code, but not which ones.

Takeaway. Fine-tuning and routing provide the strongest peak forward performance, but that specialization does not reliably transfer beyond the trained task. The SFT baselines often lose reverse-reasoning performance and frequently fall back to predicting the forward output when asked to reason in reverse. Merging sacrifices some peak forward accuracy but retains substantially more reverse-reasoning ability, and a TIES merge also preserves general code generation on every model, where single adapters lose it unpredictably.

8 Related Work

Deobfuscation and robustness. Prior work on code deobfuscation focuses primarily on recovering a cleaner representation from transformed code. Our setting instead evaluates program reasoning while the code remains obfuscated, measuring robustness to semantics-preserving changes rather than reconstruction quality.

Model merging. Model merging can preserve or strengthen knowledge shared across independently trained models, while knowledge specific to individual models is more likely to degrade during composition [16, 32]. This asymmetry has also been exploited for bias reduction and machine unlearning [31, 32]. Merge quality further depends on specialist training dynamics: memorization late in fine-tuning can produce idiosyncratic parameter updates that interfere during merging, while earlier checkpoints can compose more effectively [17].

Modular adaptation. Related work identifies similar shared-versus-specific structure within low-rank adapters. LoRA can exhibit capacity limits on high-entropy objectives, motivating architectures that route updates between low-rank and full-parameter experts [29]. Token-dependent routing has likewise been used to separate shared and subdomain-specific adapter updates [5].

Connection to our setting. These findings motivate our treatment of obfuscation specialists as containing both transform-specific adaptations and reasoning updates shared across semantics-preserving views. We test whether parameter-space composition can retain this shared program-reasoning capability while reducing dependence on individual transformation surfaces.

9 Pre-Registered Hypotheses and Their Outcomes

This project registered a hypothesis ledger before its main grids ran, and the study that resulted is not the study that was planned. We report the ledger in full, including the rows that were refuted, the row that was retired as ill-posed, and the two rows that were never resolved at all. Table 12 is that accounting. Three points in it are load-bearing for how the rest of this paper should be read.

The original discriminator was refuted early, and the paper is a consequence of that. **H1c** as registered — that a positive raw gain on the held-out obfuscator demonstrates semantic invariance — was retired within a day of the pilot, because an adapter trained on *clean* code reached the held-out family at least as well as one trained on obfuscated code (0.414 versus 0.384). The raw gain measures task acquisition, not robustness. Its replacement, **H1c-rev**, asks the same question relative to that clean-code control, and was refuted for the condition it was first tested on (−3.0 points, CI [−10.5, +3.9]). Every claim in this paper is stated against a clean-code-tuned control for this reason, and no raw difference from the untuned model is ever reported as invariance.

Modularity was registered as a positive hypothesis and came back negative. **H2b** and **H2d** predicted that per-type adapters plus a router would beat monolithic tuning, and that merges would retain most of the per-type gains. Neither held in the form registered. The decisive measurement is that the two effects offset: merging costs −3.13 points (CI [−4.78, −1.40]) while the specialists *contribute* +2.47 (CI [+1.32, +3.70]), for a net wash. **H2a**’s premise was confirmed and its consequence was not: obfuscation type is almost perfectly surface-detectable — a hard router dispatches at 1.000 accuracy — and routing on that signal still bought nothing on the held-out family. The present paper’s RQ2 is the descendant of that result, which is why it compares composition *strategies* rather than arguing for one.

Two rows were never resolved, and we do not claim them. **H3** — that the anchoring shift predicts which transfers succeed — required a regression of shift against transfer across all system × condition cells. That regression was never run. What exists is the mechanism it presupposed: re-anchoring is real and causal (**H3-causal**, confirmed by an identifier-knockout intervention, paired difference +0.104 nats, CI [+0.011, +0.209]), and an independent symbolic dead-code-elimination pass agrees with it. That supports the mechanism and does *not* establish the predictive claim, which we therefore do not make. **HA1**, on alignment to human difficulty orderings, is likewise unresolved: only the untuned correlation was read ($\rho = +0.132$, CI [−0.140, +0.381]), and the pre/post change it was about was never measured.

Standing correctness checks. The ledger was accompanied by a fixed list of silent-failure checks to be cleared before any result was trusted: that splits partition by program and never by row; that a tuned model’s outputs actually differ from the untuned one’s on the same items; that one prompt builder serves training, both evaluation paths and attention extraction; that the loss mask is verified on a real batch; that format-failure and truncation rates are reported rather than assumed; that catastrophic forgetting is measured on a general coding benchmark; and that headline numbers use the subset of programs on which every condition succeeded. Each is enforced in code or reported in this paper. The forgetting check is the one that proved hardest to make trustworthy, and Section 10 records why.

Table 12. **The pre-registered ledger, as registered and as it ended.** Registered 2026-08-05 before the main grids; verdicts are quoted from the dated log entry that decided each row. n/r marks a row that no experiment resolved — reported rather than dropped, because a ledger that only lists its successes is not a ledger.

ID	outcome	deciding evidence
H1a-trainable	SUPPORTED	self-gain +27.3 [+11.0, +43.2]
H1a family structure	SUPPORTED	within-family +33.3 vs cross +15.2
H1b cross-language	PARTIAL	JavaScript panel run; the registered contrast was not computed
H1c raw	RETIRED	confounded with task acquisition
H1c-rev control-relative	REFUTED	−3.0 [−10.5, +3.9] on the held-out family
H2a detectable $\Rightarrow$ routable	SPLIT	dispatch 1.000; no gain follows
H2b routed $\geq$ monolithic	REFUTED	as registered, at 1.5 B
H2c conditioning vs capacity	INCONCLUSIVE	recovery 0.26–0.33, in- side the reserved band
H2d merges retain $\geq$ 0.7	REFUTED	−3.13 cost against +2.47 contribution
H3 shift predicts transfer	N/R	the regression was never run
H3-causal	SUPPORTED	knockout +0.104 [+0.011, +0.209] nats
HA1 human alignment	N/R	only the untuned ρ was read

10 Threats to Validity

Held-out obfuscation family. Unseen-family generalization is evaluated through X1, a trainable sibling of the quarantined H1 obfuscator. H1 was read exactly twice, a pilot and a final pass under pre-registered hypotheses, and no method, hyperparameter or checkpoint in this paper was selected on it. On six systems trained on neither family, X1 and H1 accuracy agree at Pearson $r = 0.9992$ (mean absolute difference 0.48 points), and adapters trained on X1 lose 0.08 points when evaluated on H1. X1 therefore stands in for H1 in every experiment. This still covers only one held-out mechanism family and may not generalize to qualitatively different obfuscation mechanisms.

Uneven condition coverage. Some transformations are not applicable to every program, so paired comparisons are computed only on programs shared by the relevant conditions. The evaluated program subset therefore varies across some experiments.

Model and language scope. The study evaluates eight models, with the main experiments conducted in Python. The JavaScript set provides a limited cross-language check but does not establish generalization across programming languages.

11 Conclusion

We studied whether capabilities learned from individual obfuscation transforms can be composed to support reasoning over stacked and previously unseen obfuscations. Across eight models, pooled

and consistency-based training perform well on familiar transformation combinations but can lose capability outside the training distribution, particularly on unseen obfuscation families and the untrained reverse-reasoning task.

Parameter-space merging provides a different trade-off. By composing independently trained specialists, it preserves robustness across these out-of-distribution settings while requiring only a single adapter at inference. These results suggest that semantics-preserving transformations provide complementary training views of the same underlying computation, and that composing their learned updates can retain program-reasoning capabilities that are lost under monolithic adaptation. Evaluating this effect across additional languages and substantially different held-out obfuscation families is the primary direction for future work.

12 Data Availability

Data/code is available in project’s website [].

References

- [1] Anonymous. 2026. Attributing Gains in Paired Training: Objective or Data Direction? (2026). Under review.
- [2] Justin Chih-Yao Chen, Zifeng Wang, Hamid Palangi, Rujun Han, Sayna Ebrahimi, Long Le, Vincent Perot, Swaroop Mishra, Mohit Bansal, Chen-Yu Lee, and Tomas Pfister. 2025. Reverse Thinking Makes LLMs Stronger Reasoners. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Albuquerque, New Mexico, 8611–8630. doi:10.18653/v1/2025.naacl-long.434
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, and et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [5] Xunlei Chen, Jinyu Guo, Yuang Li, Zhaokun Wang, Yi Gong, Jie Zou, Jiwei Wei, and Wenhong Tian. 2026. ALTER: Asymmetric LoRA for Token-Entropy-Guided Unlearning of LLMs. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence*. arXiv:2603.01792 [cs.CL] <https://arxiv.org/abs/2603.01792>
- [6] CodeGemma Team. 2024. CodeGemma: Open Code Models Based on Gemma. arXiv:2406.11409 [cs.CL] <https://arxiv.org/abs/2406.11409>
- [7] Christian Collberg, Clark Thomborson, and Douglas Low. 1997. A Taxonomy of Obfuscating Transformations. *Technical Report* (1997).
- [8] Christian Collberg, Clark Thomborson, and Douglas Low. 1998. Manufacturing cheap, resilient, and stealthy opaque constructs. In *Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '98). Association for Computing Machinery, New York, NY, USA, 184–196. doi:10.1145/268946.268962
- [9] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
- [10] Gemma Team. 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL] <https://arxiv.org/abs/2503.19786>
- [11] Granite Team, IBM. 2024. Granite 3.1 Language Models. <https://github.com/ibm-granite/granite-3.1-language-models>
- [12] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [13] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. CRUX-Eval: A Benchmark for Code Reasoning, Understanding and Execution. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 16568–16621. arXiv:2401.03065 <https://proceedings.mlr.press/v235/gu24c.html>
- [14] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie LIU, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCode(BERT): Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=jLoC4ez43PZ>
- [15] Eshgin Hasanov, Md Mahadi Hassan Sibat, Santu Karmaker, and Aashish Yadavally. 2026. The Path Not Taken: Duality in Reasoning about Program Execution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 16165–16180. arXiv:2604.20917 [cs.LG] <https://aclanthology.org/2026.acl-long.735/>
- [16] Timm Hess, Gido M. van de Ven, and Tinne Tuytelaars. 2025. Forgetting of task-specific knowledge in model merging-based continual learning. arXiv:2507.23311 [cs.LG] <https://arxiv.org/abs/2507.23311>
- [17] Stefan Horoi, Guy Wolf, Eugene Belilovsky, and Gintare Karolina Dziugaite. 2026. From Memorization to Parameter Interference: How Overtraining Experts Harms Model Merging. In *Proceedings of the 43rd International*

- Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306). PMLR, Seoul, South Korea. arXiv:2506.14126 [cs.LG] <https://arxiv.org/abs/2506.14126>
- [18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=nZeVKeeFYf9>
- [19] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (Dec. 2022), 1092–1097. doi:10.1126/science.abq1158
- [20] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [21] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abul'khanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2024. StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173 [cs.SE] <https://arxiv.org/abs/2402.19173>
- [22] Anh H. N. Nguyen, Jack Le, Ilse Lahnstein Coronado, and Tien N. Nguyen. 2026. The Effect of Code Obfuscation on Human Program Comprehension. arXiv:2603.07668 [cs.SE] <https://arxiv.org/abs/2603.07668>
- [23] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. arXiv:2203.13474 [cs.LG] <https://arxiv.org/abs/2203.13474>
- [24] Serge Lionel Nikiema, Jordan Samhi, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. The Code Barrier: What LLMs Actually Understand? arXiv:2504.10557 [cs.SE] <https://arxiv.org/abs/2504.10557>
- [25] Serge Lionel Nikiema, Jordan Samhi, Micheline Bénédicte Moumoula, Albéric Euraste Djiré, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. Using Contrastive Learning to Improve Two-Way Reasoning in Large Language Models: The Obfuscation Task as a Case Study. arXiv:2509.05553 [cs.CL] <https://arxiv.org/abs/2509.05553>
- [26] Philip O’Kane, Sakir Sezer, and Kieran McLaughlin. 2011. Obfuscation: The Hidden Malware. *IEEE Security and Privacy* 9, 5 (2011), 41–47. doi:10.1109/MSP.2011.98
- [27] Xiaokai Rong, Aashish Yadavally, and Tien N. Nguyen. 2026. Post-Hoc Attention Steering of Large Language Models for Robust Code Understanding under Obfuscation. In *2026 41th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM Press.
- [28] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] <https://arxiv.org/abs/2308.12950>
- [29] Haozhan Tang, Xiuqi Zhu, Xinyin Zhang, Boxun Li, Virginia Smith, and Kevin Kuo. 2026. Beyond LoRA vs. Full Fine-Tuning: Gradient-Guided Optimizer Routing for LLM Adaptation. Version 2, 18 May 2026. arXiv:2605.07111 [cs.CL] <https://arxiv.org/abs/2605.07111>
- [30] Daniel Votipka, Seth Rabin, Kristopher Micinski, Jeffrey S. Foster, and Michelle L. Mazurek. 2019. An Observational Investigation of Reverse Engineers’ Process and Mental Models. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (CHI EA ’19). Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3290607.3313040
- [31] Haoming Xu, Shuxun Wang, Yanqiu Zhao, Yi Zhong, Ziyan Jiang, Ningyuan Zhao, Shumin Deng, Huajun Chen, and Ningyu Zhang. 2025. ZJUKLAB at SemEval-2025 Task 4: Unlearning via Model Merging. In *Proceedings of the 19th International Workshop on Semantic Evaluation (SemEval-2025)*. Association for Computational Linguistics, 566–574. <https://aclanthology.org/2025.semeval-1.79/>

[32] Kerem Zaman, Leshem Choshen, and Shashank Srivastava. 2024. Fuse to Forget: Bias Reduction and Selective Memorization through Model Fusion. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 18763–18783. doi:10.18653/v1/2024.emnlp-main.1045

A Per-model absolute tables

Table ?? reports each adaptation as a mean over a block of conditions. This appendix gives the cells those means are taken over: every method against every obfuscation, one table per model, for the eight panel models. Each cell reads *output prediction / input prediction*. Bold rows are the block means, and an accuracy is coloured where it sits above or below the untuned model in the same direction; a red number carrying a dagger is red for the other reason, its format-failure rate is above 0.25 and the accuracy beneath it is not a fair reading of the arm. CodeLlama-7B (Table 13) is the one model on which every arm exists.

Table 13. **Every method against every obfuscation, CodeLlama-7B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 1); *router* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of them. **Colour:** an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution:** any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	merge out / in
L0	0.257 / 0.285	0.302 / -	0.427 / 0.324	0.413 / 0.276	0.422 / 0.297	0.429 / 0.305	0.426 / 0.329
L1b	0.198 / 0.265	0.255 / -	0.358 / 0.251	0.388 / 0.279	0.402 / 0.294	0.381 / 0.300	0.373 / 0.297
L1r	0.207 / 0.315	0.265 / -	0.378 / 0.327	0.387 / 0.276	0.395 / 0.293	0.383 / 0.312	0.381 / 0.352
L2	0.202 / 0.282	0.272 / -	0.382 / 0.324	0.378 / 0.276	0.398 / 0.275	0.386 / 0.328	0.375 / 0.342
S1	0.168 / 0.261	0.216 / -	0.379 / 0.188†	0.389 / 0.262	0.400 / 0.280	0.395 / 0.255	0.376 / 0.308
S2	0.193 / 0.330	0.215† / -	0.389 / 0.209†	0.403 / 0.294	0.415 / 0.332	0.421 / 0.293	0.392 / 0.357
X1	0.119 / 0.257	0.154 / -	0.269 / 0.221†	0.231 / 0.237	0.286 / 0.281	0.279 / 0.206†	0.276 / 0.280
C_L1b_S1	0.141 / 0.218	0.153 / -	0.279 / 0.163†	0.345 / 0.242	0.360 / 0.270	0.325 / 0.222	0.302 / 0.280
C_L1r_S1	0.131 / 0.301	0.179 / -	0.294 / 0.209†	0.350 / 0.255	0.356 / 0.270	0.346 / 0.227†	0.301 / 0.316
C_S1_L1r	0.135 / 0.282	0.168 / -	0.285 / 0.205	0.326 / 0.267	0.349 / 0.285	0.318 / 0.242	0.301 / 0.291
C_L2_S4	0.176 / 0.291	0.239 / -	0.355 / 0.231†	0.385 / 0.260	0.392 / 0.283	0.383 / 0.309	0.363 / 0.349
C_L1r_S3	0.172 / 0.296	0.201 / -	0.360 / 0.267	0.380 / 0.278	0.387 / 0.315	0.391 / 0.303	0.368 / 0.350
C_S4_S3	0.186 / 0.315	0.223† / -	0.392 / 0.201†	0.403 / 0.282	0.422 / 0.333	0.415 / 0.287	0.398 / 0.352
C3_L1r_S3_S4	0.150 / 0.303	0.181 / -	0.340 / 0.191†	0.377 / 0.268	0.380 / 0.302	0.375 / 0.249†	0.349 / 0.336
C3_S1_S3_S4	0.128 / 0.292	0.160 / -	0.354 / 0.166†	0.369 / 0.234	0.385 / 0.313	0.377 / 0.193†	0.340 / 0.321
C3_L1r_S1_S4	0.126 / 0.299	0.167 / -	0.289 / 0.163†	0.353 / 0.229	0.354 / 0.278	0.325 / 0.181†	0.296 / 0.314
C4_L1r_S1_S3_S4	0.089 / 0.282	0.122 / -	0.278 / 0.183†	0.336 / 0.217	0.356 / 0.288	0.334 / 0.176†	0.290 / 0.296
C_L1r_X1	0.102 / 0.253	0.121 / -	0.184 / 0.210	0.199 / 0.200	0.222 / 0.250	0.189 / 0.176†	0.193 / 0.255
C_X1_S1	0.104 / 0.251	0.131 / -	0.274 / 0.139†	0.230 / 0.201	0.267 / 0.276	0.291 / 0.144†	0.290 / 0.285
C_S2_X1	0.145 / 0.311	0.184 / -	0.313 / 0.153†	0.285 / 0.217	0.332 / 0.305	0.326 / 0.209†	0.317 / 0.326
C_L1r_X1m	0.112 / 0.250	0.116 / -	0.220 / 0.220	0.232 / 0.223	0.249 / 0.250	0.227 / 0.192†	0.215 / 0.253
C_S1_X1s	0.151 / 0.277	0.206 / -	0.334 / 0.144†	0.302 / 0.195	0.355 / 0.288	0.350 / 0.168†	0.338 / 0.286
C3_L1r_S1_X1	0.104 / 0.263	0.127 / -	0.219 / 0.139†	0.222 / 0.154†	0.250 / 0.231	0.242 / 0.097†	0.232 / 0.288
single obfuscation	0.194 / 0.291	0.252 ⁴ / -	0.377 / 0.301 ³	0.389 / 0.277	0.402 / 0.295	0.393 / 0.297	0.380 / 0.331
held-out family	0.119 / 0.257	0.154 / -	0.269 / -	0.231 / 0.237	0.286 / 0.281	0.279 / -	0.276 / 0.280
depth 2, all seen	0.157 / 0.284	0.188 ⁵ / -	0.327 / 0.236 ²	0.365 / 0.264	0.378 / 0.293	0.363 / 0.273 ⁵	0.339 / 0.323
depth 3, all seen	0.135 / 0.298	0.169 / -	0.328 / -	0.366 / 0.244	0.373 / 0.298	0.359 / -	0.328 / 0.324
depth 4, all seen	0.089 / 0.282	0.122 / -	0.278 / -	0.336 / 0.217	0.356 / 0.288	0.334 / -	0.290 / 0.296
depth 2, unseen inside	0.123 / 0.268	0.152 / -	0.265 / 0.215 ²	0.249 / 0.207	0.285 / 0.274	0.277 / -	0.271 / 0.281
depth 3, unseen inside	0.104 / 0.263	0.127 / -	0.219 / -	0.222 / -	0.250 / 0.231	0.242 / -	0.232 / 0.288

Table 14. **Every method against every obfuscation, CodeLlama-13B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 1); *router* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	merge out / in
L0	0.252 / 0.285	0.278 [†] / -	0.469 / 0.269	0.446 / 0.066 [†]	0.462 / 0.231	0.477 / 0.232 [†]	0.477 / 0.339
L1b	0.222 / 0.277	0.220 [†] / -	0.397 / 0.285	0.423 / 0.066 [†]	0.438 / 0.253	0.426 / 0.304	0.400 / 0.318
L1r	0.225 / 0.327	0.233 [†] / -	0.415 / 0.347	0.420 / 0.051 [†]	0.439 / 0.229	0.428 / 0.274	0.425 / 0.366
L2	0.228 / 0.279	0.244 [†] / -	0.407 / 0.232 [†]	0.417 / 0.026 [†]	0.432 / 0.206	0.424 / 0.152 [†]	0.421 / 0.318
S1	0.213 / 0.304	0.216 / -	0.408 / 0.341	0.413 / 0.059 [†]	0.435 / 0.199	0.435 / 0.294	0.435 / 0.363
S2	0.184 / 0.293	0.187 [†] / -	0.422 / 0.317	0.445 / 0.038 [†]	0.469 / 0.238	0.453 / 0.288 [†]	0.441 / 0.365
X1	0.149 / 0.258	0.141 [†] / -	0.294 / 0.274	0.254 / 0.016 [†]	0.292 / 0.219	0.313 / 0.246	0.306 / 0.298
C_L1b_S1	0.183 / 0.253	0.159 / -	0.318 / 0.326	0.367 / 0.059 [†]	0.375 / 0.201	0.370 / 0.298	0.329 / 0.341
C_L1r_S1	0.175 / 0.325	0.175 / -	0.318 / 0.360	0.371 / 0.070 [†]	0.376 / 0.203	0.371 / 0.303	0.356 / 0.368
C_S1_L1r	0.172 / 0.323	0.172 / -	0.315 / 0.351	0.359 / 0.086 [†]	0.363 / 0.230	0.370 / 0.310	0.341 / 0.376
C_L2_S4	0.200 / 0.274	0.187 [†] / -	0.389 / 0.186 [†]	0.422 / 0.020 [†]	0.432 / 0.168 [†]	0.421 / 0.132 [†]	0.404 / 0.321
C_L1r_S3	0.186 / 0.305	0.194 [†] / -	0.407 / 0.361	0.409 / 0.041 [†]	0.434 / 0.220	0.428 / 0.325	0.417 / 0.366
C_S4_S3	0.192 / 0.293	0.211 [†] / -	0.424 / 0.326	0.436 / 0.026 [†]	0.459 / 0.239	0.464 / 0.288 [†]	0.443 / 0.363
C3_L1r_S3_S4	0.166 / 0.310	0.169 [†] / -	0.382 / 0.337	0.410 / 0.032 [†]	0.429 / 0.224	0.416 / 0.283 [†]	0.394 / 0.353
C3_S1_S3_S4	0.147 / 0.295	0.158 / -	0.367 / 0.329	0.408 / 0.035 [†]	0.423 / 0.235	0.419 / 0.312	0.387 / 0.347
C3_L1r_S1_S4	0.163 / 0.316	0.156 / -	0.314 / 0.353	0.367 / 0.061 [†]	0.367 / 0.211	0.362 / 0.327	0.346 / 0.370
C4_L1r_S1_S3_S4	0.117 / 0.310	0.117 / -	0.300 / 0.333	0.353 / 0.049 [†]	0.360 / 0.222	0.363 / 0.332	0.320 / 0.353
C_L1r_X1	0.123 / 0.281	0.115 [†] / -	0.199 / 0.254	0.213 / 0.016 [†]	0.236 / 0.170 [†]	0.227 / 0.187 [†]	0.217 / 0.266
C_X1_S1	0.138 / 0.289	0.135 / -	0.295 / 0.264	0.270 / 0.007 [†]	0.307 / 0.201 [†]	0.320 / 0.233	0.310 / 0.319
C_S2_X1	0.160 / 0.271	0.167 [†] / -	0.335 / 0.279	0.334 / 0.008 [†]	0.356 / 0.204 [†]	0.360 / 0.217 [†]	0.353 / 0.329
C_L1r_X1m	0.138 / 0.291	0.127 [†] / -	0.246 / 0.259	0.248 / 0.020 [†]	0.279 / 0.179 [†]	0.267 / 0.199 [†]	0.251 / 0.283
C_S1_X1s	0.179 / 0.301	0.166 / -	0.364 / 0.290	0.345 / 0.025 [†]	0.360 / 0.199 [†]	0.390 / 0.267	0.376 / 0.317
C3_L1r_S1_X1	0.128 / 0.304	0.107 [†] / -	0.233 / 0.293	0.234 / 0.026 [†]	0.249 / 0.185 [†]	0.254 / 0.203 [†]	0.251 / 0.313
single obfuscation	0.215 / 0.296	0.216 ¹ / -	0.410 / 0.323 ⁴	0.423 / -	0.442 / 0.225	0.433 / 0.291 ³	0.424 / 0.346
held-out family	0.149 / 0.258	- / -	0.294 / 0.274	0.254 / -	0.292 / 0.219	0.313 / 0.246	0.306 / 0.298
depth 2, all seen	0.185 / 0.295	0.168 ³ / -	0.362 / 0.345 ⁵	0.394 / -	0.407 / 0.219 ⁵	0.404 / 0.309 ⁴	0.381 / 0.356
depth 3, all seen	0.159 / 0.307	0.157 ² / -	0.354 / 0.340	0.395 / -	0.406 / 0.223	0.399 / 0.320 ²	0.375 / 0.357
depth 4, all seen	0.117 / 0.310	0.117 / -	0.300 / 0.333	0.353 / -	0.360 / 0.222	0.363 / 0.332	0.320 / 0.353
depth 2, unseen inside	0.148 / 0.286	0.151 ² / -	0.288 / 0.269	0.282 / -	0.307 / -	0.313 / 0.250 ²	0.301 / 0.303
depth 3, unseen inside	0.128 / 0.304	- / -	0.233 / 0.293	0.234 / -	0.249 / -	0.254 / -	0.251 / 0.313

Table 15. **Every method against every obfuscation, CodeLlama-34B.** Each cell is *output prediction* / *input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 1); *router* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of them. **Colour:** an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution:** any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	merge out / in
L0	0.254 [†] / 0.363	0.178 [†] / -	0.520 / 0.260	0.494 / 0.200 [†]	0.525 / 0.284	0.526 / 0.239 [†]	0.527 / 0.382
L1b	0.205 / 0.327	0.077 [†] / -	0.417 / 0.253	0.469 / 0.174 [†]	0.494 / 0.287	0.472 / 0.259 [†]	0.429 / 0.360
L1r	0.220 / 0.339	0.117 [†] / -	0.469 / 0.255	0.457 / 0.135 [†]	0.487 / 0.256	0.481 / 0.170 [†]	0.467 / 0.373
L2	0.231 / 0.357	0.109 [†] / -	0.466 / 0.280	0.465 / 0.179 [†]	0.478 / 0.276	0.475 / 0.200 [†]	0.463 / 0.378
S1	0.222 / 0.323	0.079 [†] / -	0.472 / 0.305	0.467 / 0.178 [†]	0.496 / 0.264	0.484 / 0.241 [†]	0.456 / 0.390
S2	0.191 / 0.350	0.040 [†] / -	0.484 / 0.354	0.484 / 0.219 [†]	0.525 / 0.306	0.509 / 0.293 [†]	0.484 / 0.437
X1	0.135 / 0.303	0.050 [†] / -	0.325 / 0.253	0.298 / 0.118 [†]	0.334 / 0.249	0.344 / 0.114 [†]	0.328 / 0.338
C_L1b_S1	0.156 / 0.257	0.033 [†] / -	0.322 / 0.272	0.399 / 0.144 [†]	0.421 / 0.258	0.390 / 0.221 [†]	0.331 / 0.358
C_L1r_S1	0.184 / 0.304	0.063 [†] / -	0.351 / 0.285	0.393 / 0.132 [†]	0.427 / 0.200	0.415 / 0.145 [†]	0.362 / 0.375
C_S1_L1r	0.180 / 0.292	0.053 [†] / -	0.367 / 0.307	0.405 / 0.142 [†]	0.425 / 0.216	0.387 / 0.163 [†]	0.377 / 0.371
C_L2_S4	0.191 / 0.344	0.045 [†] / -	0.431 / 0.291	0.454 / 0.170 [†]	0.478 / 0.273	0.452 / 0.243 [†]	0.436 / 0.384
C_L1r_S3	0.169 / 0.333	0.041 [†] / -	0.451 / 0.327	0.454 / 0.167 [†]	0.482 / 0.256	0.471 / 0.197 [†]	0.453 / 0.419
C_S4_S3	0.199 / 0.352	0.041 [†] / -	0.482 / 0.345	0.487 / 0.218 [†]	0.521 / 0.301	0.505 / 0.276 [†]	0.488 / 0.442
C3_L1r_S3_S4	0.148 / 0.325	0.017 [†] / -	0.416 / 0.330	0.452 / 0.182 [†]	0.473 / 0.251	0.449 / 0.201 [†]	0.425 / 0.427
C3_S1_S3_S4	0.128 / 0.330	0.042 [†] / -	0.428 / 0.323	0.454 / 0.208 [†]	0.472 / 0.288	0.469 / 0.235 [†]	0.442 / 0.413
C3_L1r_S1_S4	0.147 / 0.291	0.041 [†] / -	0.338 / 0.311	0.390 / 0.121 [†]	0.412 / 0.199	0.391 / 0.134 [†]	0.354 / 0.381
C4_L1r_S1_S3_S4	0.105 / 0.283	0.033 [†] / -	0.316 / 0.333	0.372 / 0.164 [†]	0.413 / 0.214	0.380 / 0.175 [†]	0.351 / 0.386
C_L1r_X1	0.115 / 0.275	0.044 [†] / -	0.226 / 0.240	0.249 / 0.060 [†]	0.278 / 0.162 [†]	0.263 / 0.030 [†]	0.236 / 0.299
C_X1_S1	0.125 / 0.298	0.064 [†] / -	0.317 / 0.275	0.290 / 0.089 [†]	0.339 / 0.216 [†]	0.342 / 0.091 [†]	0.317 / 0.345
C_S2_X1	0.176 / 0.378	0.099 [†] / -	0.372 / 0.366	0.355 / 0.196 [†]	0.403 / 0.288	0.395 / 0.178 [†]	0.382 / 0.414
C_L1r_X1m	0.128 / 0.269	0.052 [†] / -	0.261 / 0.230	0.275 / 0.077 [†]	0.294 / 0.158 [†]	0.296 / 0.030 [†]	0.279 / 0.308
C_S1_X1s	0.176 / 0.346	0.077 [†] / -	0.390 / 0.302	0.382 / 0.141 [†]	0.415 / 0.268	0.397 / 0.115 [†]	0.387 / 0.418
C3_L1r_S1_X1	0.118 / 0.304	0.048 [†] / -	0.256 / 0.296	0.262 / 0.084 [†]	0.297 / 0.188 [†]	0.271 / 0.021 [†]	0.267 / 0.357
single obfuscation	0.214 / 0.339	- / -	0.462 / 0.289	0.469 / -	0.496 / 0.278	0.484 / -	0.460 / 0.388
held-out family	0.135 / 0.303	- / -	0.325 / 0.253	0.298 / -	0.334 / 0.249	0.344 / -	0.328 / 0.338
depth 2, all seen	0.180 / 0.314	- / -	0.401 / 0.305	0.432 / -	0.459 / 0.251	0.437 / -	0.408 / 0.391
depth 3, all seen	0.141 / 0.315	- / -	0.394 / 0.322	0.432 / -	0.452 / 0.246	0.436 / -	0.407 / 0.407
depth 4, all seen	0.105 / 0.283	- / -	0.316 / 0.333	0.372 / -	0.413 / 0.214	0.380 / -	0.351 / 0.386
depth 2, unseen inside	0.144 / 0.313	- / -	0.313 / 0.283	0.310 / -	0.346 / 0.278 ²	0.338 / -	0.320 / 0.357
depth 3, unseen inside	0.118 / 0.304	- / -	0.256 / 0.296	0.262 / -	0.297 / -	0.271 / -	0.267 / 0.357

Table 16. **Every method against every obfuscation, Llama-3.1-8B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 1); *router* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	merge out / in
L0	0.258 / 0.409	0.320 / -	0.448 / 0.335	0.426 / 0.320	0.445 / 0.332	0.475 / 0.338	0.462 / 0.414
L1b	0.222 / 0.371	0.276 / -	0.361 / 0.297	0.387 / 0.280	0.407 / 0.285	0.422 / 0.318	0.386 / 0.383
L1r	0.242 / 0.373	0.284 / -	0.387 / 0.312	0.387 / 0.290	0.405 / 0.297	0.407 / 0.311	0.398 / 0.372
L2	0.215 / 0.376	0.278 / -	0.391 / 0.297	0.386 / 0.284	0.409 / 0.294	0.425 / 0.311	0.396 / 0.364
S1	0.213 / 0.375	0.256 / -	0.395 / 0.325	0.391 / 0.285	0.413 / 0.291	0.428 / 0.316	0.412 / 0.411
S2	0.205 / 0.378	0.272 / -	0.413 / 0.345	0.413 / 0.326	0.445 / 0.325	0.456 / 0.366	0.437 / 0.435
X1	0.132 / 0.263	0.161 / -	0.249 / 0.267	0.222 / 0.211	0.255 / 0.228	0.280 / 0.258	0.266 / 0.311
C_L1b_S1	0.152 / 0.287	0.185 / -	0.279 / 0.253	0.338 / 0.274	0.368 / 0.258	0.369 / 0.306	0.307 / 0.352
C_L1r_S1	0.162 / 0.297	0.220 / -	0.303 / 0.281	0.352 / 0.280	0.363 / 0.283	0.366 / 0.293	0.328 / 0.350
C_S1_L1r	0.159 / 0.303	0.208 / -	0.277 / 0.269	0.338 / 0.274	0.354 / 0.274	0.347 / 0.293	0.306 / 0.356
C_L2_S4	0.205 / 0.320	0.257 / -	0.349 / 0.275	0.386 / 0.269	0.401 / 0.286	0.413 / 0.313	0.383 / 0.388
C_L1r_S3	0.198 / 0.342	0.266 / -	0.367 / 0.297	0.382 / 0.299	0.398 / 0.303	0.407 / 0.320	0.387 / 0.375
C_S4_S3	0.214 / 0.367	0.269 / -	0.421 / 0.333	0.411 / 0.313	0.444 / 0.329	0.459 / 0.363	0.437 / 0.417
C3_L1r_S3_S4	0.183 / 0.309	0.233 / -	0.329 / 0.271	0.374 / 0.303	0.391 / 0.300	0.399 / 0.317	0.370 / 0.372
C3_S1_S3_S4	0.160 / 0.353	0.220 / -	0.359 / 0.311	0.376 / 0.289	0.393 / 0.281	0.415 / 0.311	0.377 / 0.408
C3_L1r_S1_S4	0.160 / 0.302	0.208 / -	0.279 / 0.270	0.342 / 0.262	0.358 / 0.271	0.353 / 0.276	0.319 / 0.341
C4_L1r_S1_S3_S4	0.140 / 0.281	0.191 / -	0.263 / 0.234	0.335 / 0.280	0.358 / 0.279	0.342 / 0.274	0.310 / 0.340
C_L1r_X1	0.100 / 0.218	0.118 / -	0.165 / 0.220	0.181 / 0.193	0.207 / 0.239	0.204 / 0.230	0.184 / 0.275
C_X1_S1	0.130 / 0.291	0.179 / -	0.271 / 0.283	0.225 / 0.210	0.253 / 0.220	0.294 / 0.269	0.296 / 0.332
C_S2_X1	0.170 / 0.323	0.212 / -	0.296 / 0.288	0.284 / 0.229	0.317 / 0.241	0.336 / 0.284	0.317 / 0.350
C_L1r_X1m	0.115 / 0.232	0.148 / -	0.184 / 0.210	0.216 / 0.217	0.232 / 0.245	0.238 / 0.225	0.217 / 0.274
C_S1_X1s	0.193 / 0.357	0.231 / -	0.341 / 0.287	0.299 / 0.205	0.335 / 0.212	0.362 / 0.276	0.349 / 0.376
C3_L1r_S1_X1	0.105 / 0.268	0.156 / -	0.194 / 0.234	0.207 / 0.188	0.224 / 0.194	0.238 / 0.241	0.215 / 0.305
single obfuscation	0.219 / 0.375	0.273 / -	0.390 / 0.315	0.393 / 0.293	0.416 / 0.299	0.427 / 0.324	0.406 / 0.393
held-out family	0.132 / 0.263	0.161 / -	0.249 / 0.267	0.222 / 0.211	0.255 / 0.228	0.280 / 0.258	0.266 / 0.311
depth 2, all seen	0.182 / 0.319	0.234 / -	0.332 / 0.284	0.368 / 0.285	0.388 / 0.289	0.394 / 0.315	0.358 / 0.373
depth 3, all seen	0.168 / 0.321	0.221 / -	0.322 / 0.284	0.364 / 0.285	0.380 / 0.284	0.389 / 0.301	0.355 / 0.374
depth 4, all seen	0.140 / 0.281	0.191 / -	0.263 / 0.234	0.335 / 0.280	0.358 / 0.279	0.342 / 0.274	0.310 / 0.340
depth 2, unseen inside	0.142 / 0.284	0.177 / -	0.252 / 0.258	0.241 / 0.211	0.269 / 0.231	0.287 / 0.257	0.273 / 0.321
depth 3, unseen inside	0.105 / 0.268	0.156 / -	0.194 / 0.234	0.207 / 0.188	0.224 / 0.194	0.238 / 0.241	0.215 / 0.305

Table 17. **Every method against every obfuscation, StarCoder2-15B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 1); *router* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	merge out / in
L0	0.000 [†] / 0.366	0.008 [†] / -	0.542 / 0.411	0.529 / 0.311	0.532 / 0.399	0.546 / 0.435	0.539 / 0.452
L1b	0.000 [†] / 0.216 [†]	0.000 [†] / -	0.431 / 0.316	0.487 / 0.301	0.510 / 0.395	0.489 / 0.382	0.426 / 0.364
L1r	0.000 [†] / 0.282 [†]	0.000 [†] / -	0.483 / 0.354	0.485 / 0.273	0.509 / 0.369	0.492 / 0.368	0.476 / 0.423
L2	0.000 [†] / 0.355	0.002 [†] / -	0.479 / 0.391	0.486 / 0.297	0.503 / 0.379	0.493 / 0.406	0.466 / 0.421
S1	0.000 [†] / 0.235 [†]	0.010 [†] / -	0.508 / 0.361	0.487 / 0.257 [†]	0.501 / 0.364	0.510 / 0.377	0.487 / 0.421
S2	0.000 [†] / 0.238 [†]	0.023 [†] / -	0.512 / 0.359	0.514 / 0.300	0.539 / 0.384	0.533 / 0.404	0.518 / 0.440
X1	0.000 [†] / 0.235 [†]	0.009 [†] / -	0.347 / 0.254 [†]	0.320 / 0.177 [†]	0.343 / 0.281	0.353 / 0.261 [†]	0.343 / 0.328
C_L1b_S1	0.000 [†] / 0.122 [†]	0.001 [†] / -	0.360 / 0.254 [†]	0.416 / 0.211 [†]	0.456 / 0.317	0.417 / 0.297	0.354 / 0.324
C_L1r_S1	0.000 [†] / 0.165 [†]	0.000 [†] / -	0.393 / 0.301	0.423 / 0.214 [†]	0.440 / 0.303	0.419 / 0.305	0.389 / 0.388
C_S1_L1r	0.000 [†] / 0.136 [†]	0.000 [†] / -	0.377 / 0.311	0.406 / 0.183 [†]	0.431 / 0.303	0.418 / 0.309	0.387 / 0.374
C_L2_S4	0.000 [†] / 0.215 [†]	0.005 [†] / -	0.454 / 0.342	0.470 / 0.260 [†]	0.508 / 0.353	0.485 / 0.378	0.438 / 0.395
C_L1r_S3	0.000 [†] / 0.243 [†]	0.000 [†] / -	0.473 / 0.300	0.483 / 0.282	0.504 / 0.368	0.490 / 0.323	0.459 / 0.383
C_S4_S3	0.000 [†] / 0.243 [†]	0.028 [†] / -	0.511 / 0.368	0.516 / 0.301	0.539 / 0.386	0.525 / 0.389	0.506 / 0.448
C3_L1r_S3_S4	0.000 [†] / 0.182 [†]	0.004 [†] / -	0.448 / 0.294	0.484 / 0.242 [†]	0.503 / 0.339	0.475 / 0.315	0.436 / 0.386
C3_S1_S3_S4	0.000 [†] / 0.191 [†]	0.019 [†] / -	0.476 / 0.337	0.465 / 0.235 [†]	0.493 / 0.333	0.480 / 0.345	0.464 / 0.406
C3_L1r_S1_S4	0.000 [†] / 0.096 [†]	0.002 [†] / -	0.370 / 0.293	0.417 / 0.209 [†]	0.439 / 0.270	0.413 / 0.297	0.376 / 0.353
C4_L1r_S1_S3_S4	0.000 [†] / 0.104 [†]	0.001 [†] / -	0.357 / 0.289	0.412 / 0.206 [†]	0.437 / 0.273	0.401 / 0.271	0.358 / 0.358
C_L1r_X1	0.000 [†] / 0.123 [†]	0.002 [†] / -	0.248 / 0.165 [†]	0.274 / 0.165 [†]	0.286 / 0.205 [†]	0.278 / 0.168 [†]	0.259 / 0.262
C_X1_S1	0.000 [†] / 0.194 [†]	0.022 [†] / -	0.357 / 0.259 [†]	0.322 / 0.127 [‡]	0.342 / 0.243 [†]	0.385 / 0.238 [†]	0.371 / 0.355
C_S2_X1	0.000 [†] / 0.184 [†]	0.014 [†] / -	0.403 / 0.261 [†]	0.375 / 0.196 [†]	0.397 / 0.268 [†]	0.413 / 0.276 [†]	0.403 / 0.384
C_L1r_X1m	0.000 [†] / 0.179 [†]	0.003 [†] / -	0.307 / 0.191 [†]	0.323 / 0.178 [†]	0.338 / 0.234	0.328 / 0.180 [†]	0.317 / 0.267
C_S1_X1s	0.000 [†] / 0.258 [†]	0.044 [†] / -	0.434 / 0.291 [†]	0.408 / 0.176 [†]	0.434 / 0.286 [†]	0.443 / 0.272 [†]	0.427 / 0.384
C3_L1r_S1_X1	0.000 [†] / 0.153 [†]	0.002 [†] / -	0.278 / 0.191 [†]	0.298 / 0.107 [‡]	0.302 / 0.167 [†]	0.306 / 0.146 [†]	0.286 / 0.323
single obfuscation	- / 0.355 ¹	- / -	0.482 / 0.356	0.492 / 0.293 ⁴	0.512 / 0.378	0.503 / 0.387	0.475 / 0.414
held-out family	- / -	- / -	0.347 / -	0.320 / -	0.343 / 0.281	0.353 / -	0.343 / 0.328
depth 2, all seen	- / -	- / -	0.428 / 0.325 ⁵	0.452 / 0.291 ²	0.480 / 0.338	0.459 / 0.334	0.422 / 0.385
depth 3, all seen	- / -	- / -	0.431 / 0.308	0.455 / -	0.478 / 0.314	0.456 / 0.319	0.425 / 0.382
depth 4, all seen	- / -	- / -	0.357 / 0.289	0.412 / -	0.437 / 0.273	0.401 / 0.271	0.358 / 0.358
depth 2, unseen inside	- / -	- / -	0.350 / -	0.341 / -	0.359 / 0.234 ¹	0.369 / -	0.355 / 0.330
depth 3, unseen inside	- / -	- / -	0.278 / -	0.298 / -	0.302 / -	0.306 / -	0.286 / 0.323

Table 18. **Every method against every obfuscation, Gemma-3-12B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 1); *router* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	merge out / in
L0	0.334 [†] / 0.453	0.340 [†] / -	0.520 / 0.440	0.507 / 0.447	0.502 / 0.355	0.530 / 0.474	0.523 / 0.481
L1b	0.262 [†] / 0.405	0.274 [†] / -	0.441 / 0.353	0.475 / 0.427	0.488 / 0.309 [†]	0.481 / 0.436	0.464 / 0.425
L1r	0.273 [†] / 0.420	0.283 [†] / -	0.459 / 0.394	0.463 / 0.411	0.472 / 0.280 [†]	0.477 / 0.433	0.474 / 0.440
L2	0.269 [†] / 0.403	0.281 [†] / -	0.456 / 0.378	0.471 / 0.405	0.478 / 0.295 [†]	0.481 / 0.424	0.465 / 0.420
S1	0.269 [†] / 0.424	0.289 [†] / -	0.456 / 0.375	0.457 / 0.421	0.483 / 0.256 [†]	0.496 / 0.446	0.479 / 0.435
S2	0.290 [†] / 0.443	0.308 [†] / -	0.500 / 0.444	0.503 / 0.457	0.504 / 0.277 [†]	0.527 / 0.486	0.513 / 0.484
X1	0.152 / 0.333	0.183 / -	0.310 / 0.305	0.263 / 0.333	0.289 / 0.199 [†]	0.314 / 0.351	0.311 / 0.371
C_L1b_S1	0.195 [†] / 0.349	0.197 [†] / -	0.333 / 0.317	0.406 / 0.366	0.420 / 0.214 [†]	0.414 / 0.375	0.359 / 0.396
C_L1r_S1	0.200 [†] / 0.356	0.204 [†] / -	0.336 / 0.319	0.389 / 0.379	0.416 / 0.203 [†]	0.398 / 0.386	0.362 / 0.386
C_S1_L1r	0.181 [†] / 0.337	0.193 [†] / -	0.334 / 0.319	0.393 / 0.379	0.414 / 0.207 [†]	0.389 / 0.403	0.338 / 0.405
C_L2_S4	0.249 [†] / 0.387	0.262 [†] / -	0.438 / 0.381	0.460 / 0.416	0.469 / 0.222 [†]	0.459 / 0.451	0.452 / 0.430
C_L1r_S3	0.260 [†] / 0.427	0.274 [†] / -	0.438 / 0.402	0.462 / 0.423	0.469 / 0.254 [†]	0.461 / 0.445	0.450 / 0.443
C_S4_S3	0.297 [†] / 0.434	0.308 [†] / -	0.491 / 0.438	0.506 / 0.462	0.503 / 0.274 [†]	0.518 / 0.493	0.496 / 0.472
C3_L1r_S3_S4	0.232 [†] / 0.398	0.256 [†] / -	0.419 / 0.386	0.454 / 0.420	0.467 / 0.223 [†]	0.455 / 0.446	0.440 / 0.441
C3_S1_S3_S4	0.220 [†] / 0.387	0.256 [†] / -	0.425 / 0.354	0.452 / 0.419	0.465 / 0.240 [†]	0.482 / 0.456	0.449 / 0.430
C3_L1r_S1_S4	0.176 [†] / 0.347	0.191 [†] / -	0.320 / 0.302	0.390 / 0.376	0.415 / 0.181 [†]	0.395 / 0.376	0.341 / 0.377
C4_L1r_S1_S3_S4	0.170 [†] / 0.329	0.172 [†] / -	0.302 / 0.293	0.394 / 0.364	0.413 / 0.180 [†]	0.383 / 0.383	0.332 / 0.370
C_L1r_X1	0.102 / 0.258	0.126 / -	0.195 / 0.242	0.206 / 0.279	0.224 / 0.121 [†]	0.213 / 0.279	0.203 / 0.274
C_X1_S1	0.172 / 0.348	0.193 / -	0.317 / 0.282	0.271 / 0.331	0.296 / 0.132 [†]	0.333 / 0.360	0.312 / 0.375
C_S2_X1	0.201 / 0.360	0.222 / -	0.368 / 0.339	0.349 / 0.357	0.364 / 0.210 [†]	0.389 / 0.402	0.368 / 0.395
C_L1r_X1m	0.108 [†] / 0.273	0.115 [†] / -	0.220 / 0.269	0.235 / 0.323	0.244 / 0.146 [†]	0.250 / 0.307	0.231 / 0.285
C_S1_X1s	0.255 / 0.412	0.275 / -	0.395 / 0.371	0.374 / 0.383	0.395 / 0.178 [†]	0.421 / 0.429	0.393 / 0.438
C3_L1r_S1_X1	0.136 / 0.279	0.147 [†] / -	0.225 / 0.226	0.236 / 0.270	0.258 / 0.115 [†]	0.245 / 0.297	0.246 / 0.300
single obfuscation	- / 0.419	- / -	0.462 / 0.389	0.474 / 0.424	0.485 / -	0.492 / 0.445	0.479 / 0.441
held-out family	0.152 / 0.333	0.183 / -	0.310 / 0.305	0.263 / 0.333	0.289 / -	0.314 / 0.351	0.311 / 0.371
depth 2, all seen	- / 0.382	- / -	0.395 / 0.363	0.436 / 0.404	0.448 / -	0.440 / 0.425	0.409 / 0.422
depth 3, all seen	- / 0.377	- / -	0.388 / 0.348	0.432 / 0.405	0.449 / -	0.444 / 0.426	0.410 / 0.416
depth 4, all seen	- / 0.329	- / -	0.302 / 0.293	0.394 / 0.364	0.413 / -	0.383 / 0.383	0.332 / 0.370
depth 2, unseen inside	0.182 [‡] / 0.330	0.204 [‡] / -	0.299 / 0.301	0.287 / 0.335	0.305 / -	0.321 / 0.356	0.301 / 0.353
depth 3, unseen inside	0.136 / 0.279	- / -	0.225 / 0.226	0.236 / 0.270	0.258 / -	0.245 / 0.297	0.246 / 0.300

Table 19. **Every method against every obfuscation, CodeGemma-7B.** Each cell is *output prediction* / *input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 1); *router* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of them. **Colour:** an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution:** any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	merge out / in
L0	0.237 / 0.266	0.267 [†] / -	0.465 / 0.364	0.439 / 0.324	0.445 / 0.098 [‡]	0.474 / 0.323	0.465 / 0.329
L1b	0.185 / 0.268	0.209 [†] / -	0.382 / 0.309	0.425 / 0.298	0.435 / 0.067 [‡]	0.443 / 0.307	0.391 / 0.301
L1r	0.213 / 0.251	0.225 [†] / -	0.399 / 0.333	0.417 / 0.301	0.417 / 0.107 [‡]	0.426 / 0.294	0.411 / 0.303
L2	0.208 / 0.239	0.230 [†] / -	0.412 / 0.327	0.422 / 0.291	0.422 / 0.078 [‡]	0.440 / 0.287	0.406 / 0.299
S1	0.195 / 0.234	0.212 / -	0.405 / 0.357	0.415 / 0.337	0.421 / 0.125 [‡]	0.457 / 0.320	0.414 / 0.312
S2	0.198 / 0.270	0.242 [†] / -	0.450 / 0.372	0.429 / 0.351	0.448 / 0.126 [‡]	0.479 / 0.359	0.448 / 0.330
X1	0.143 / 0.190	0.157 / -	0.271 / 0.260	0.251 / 0.217	0.247 / 0.049 [‡]	0.301 / 0.241	0.274 / 0.264
C_L1b_S1	0.148 / 0.229	0.163 / -	0.286 / 0.274	0.372 / 0.276	0.368 / 0.089 [‡]	0.365 / 0.267	0.302 / 0.297
C_L1r_S1	0.144 / 0.170	0.144 / -	0.315 / 0.289	0.373 / 0.290	0.367 / 0.086 [‡]	0.364 / 0.278	0.318 / 0.271
C_S1_L1r	0.132 / 0.168	0.149 / -	0.288 / 0.284	0.363 / 0.316	0.361 / 0.099 [‡]	0.350 / 0.286	0.300 / 0.281
C_L2_S4	0.176 / 0.239	0.208 [†] / -	0.391 / 0.318	0.423 / 0.299	0.421 / 0.070 [‡]	0.431 / 0.314	0.391 / 0.295
C_L1r_S3	0.183 / 0.223	0.216 [†] / -	0.396 / 0.326	0.415 / 0.314	0.410 / 0.150 [†]	0.430 / 0.314	0.393 / 0.293
C_S4_S3	0.202 / 0.257	0.235 [†] / -	0.448 / 0.372	0.430 / 0.357	0.441 / 0.130 [‡]	0.475 / 0.354	0.445 / 0.348
C3_L1r_S3_S4	0.157 / 0.187	0.182 [†] / -	0.376 / 0.318	0.407 / 0.312	0.411 / 0.083 [‡]	0.427 / 0.312	0.372 / 0.294
C3_S1_S3_S4	0.158 / 0.231	0.185 / -	0.367 / 0.338	0.395 / 0.332	0.405 / 0.138 [‡]	0.430 / 0.338	0.378 / 0.306
C3_L1r_S1_S4	0.126 / 0.173	0.137 / -	0.289 / 0.274	0.367 / 0.286	0.366 / 0.064 [‡]	0.367 / 0.269	0.303 / 0.267
C4_L1r_S1_S3_S4	0.124 / 0.150	0.125 [†] / -	0.271 / 0.267	0.361 / 0.290	0.359 / 0.081 [‡]	0.360 / 0.279	0.303 / 0.270
C_L1r_X1	0.118 / 0.133 [†]	0.110 / -	0.194 / 0.208	0.212 / 0.174	0.197 / 0.036 [‡]	0.205 / 0.194	0.184 / 0.202
C_X1_S1	0.144 / 0.202	0.152 / -	0.285 / 0.303	0.261 / 0.209	0.247 / 0.034 [‡]	0.313 / 0.247	0.288 / 0.280
C_S2_X1	0.188 / 0.227	0.198 / -	0.344 / 0.300	0.323 / 0.241	0.326 / 0.031 [‡]	0.364 / 0.289	0.340 / 0.293
C_L1r_X1m	0.130 / 0.146 [†]	0.112 / -	0.214 / 0.231	0.256 / 0.211	0.234 / 0.049 [‡]	0.228 / 0.209	0.201 / 0.230
C_S1_X1s	0.186 / 0.230	0.210 / -	0.338 / 0.322	0.351 / 0.230	0.339 / 0.037 [‡]	0.393 / 0.260	0.358 / 0.302
C3_L1r_S1_X1	0.134 / 0.168	0.118 / -	0.218 / 0.230	0.241 / 0.140 [†]	0.246 / 0.013 [‡]	0.258 / 0.191	0.224 / 0.232
single obfuscation	0.200 / 0.252	0.212 ¹ / -	0.410 / 0.340	0.422 / 0.315	0.429 / -	0.449 / 0.313	0.414 / 0.309
held-out family	0.143 / 0.190	0.157 / -	0.271 / 0.260	0.251 / 0.217	0.247 / -	0.301 / 0.241	0.274 / 0.264
depth 2, all seen	0.164 / 0.214	0.152 ³ / -	0.354 / 0.310	0.396 / 0.308	0.395 / -	0.403 / 0.302	0.358 / 0.298
depth 3, all seen	0.147 / 0.197	0.161 ² / -	0.344 / 0.310	0.390 / 0.310	0.394 / -	0.408 / 0.306	0.351 / 0.289
depth 4, all seen	0.124 / 0.150	- / -	0.271 / 0.267	0.361 / 0.290	0.359 / -	0.360 / 0.279	0.303 / 0.270
depth 2, unseen inside	0.153 / 0.220 ³	0.156 / -	0.275 / 0.273	0.280 / 0.213	0.269 / -	0.301 / 0.240	0.274 / 0.261
depth 3, unseen inside	0.134 / 0.168	0.118 / -	0.218 / 0.230	0.241 / -	0.246 / -	0.258 / 0.191	0.224 / 0.232

Table 20. **Every method against every obfuscation, Granite-3.1-8B.** Each cell is *output prediction / input prediction*: raw exact-match accuracy on the forward task (predict the return value of the code as shown) and, after the slash, on the backward task (produce a call that returns the shown value), on the held-out programs of that condition. *base* is the untuned model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all six training conditions pooled; *KL* adds a KL-consistency term to a frozen clean-code teacher (Eq. 1); *router* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of them. **Colour**: an accuracy **above** or **below** base in the same direction on the same condition; base is the reference, and an uncoloured number means no comparison was available rather than a tie. A **red number carrying † or ‡** is red for the other reason — its format-failure rate is above 0.25, so the accuracy beneath it is not a fair reading of the arm — and those cells are excluded from the bold means. **Bold rows** are means over the group named, equally weighted per condition; a superscript gives the number of readable cells averaged when it is short of the group. The family adapter (tuned_X1) is deliberately not a column: it is trained on the held-out family’s sibling, so on exactly the rows that matter most it is the one arm for which that family is not held out; the **held-out family** row names a group of conditions, not an arm. **Backward is graded by execution**: any call returning the gold value is correct, because inversion is many-to-one. †: format-failure rate above 0.25, mostly replies that do not meet the output contract. ‡: above 0.25 backward, but most replies are exactly the gold *forward* answer — the arm answered the other question, which is a result rather than a broken template. ‘-’: not run.

Condition	base out / in	ICL out / in	clean LoRA out / in	breadth out / in	KL out / in	router out / in	merge out / in
L0	0.285 / 0.291	0.293 / -	0.430 / 0.206 [†]	0.416 / 0.261	0.396 / 0.117 [†]	0.438 / 0.268	0.433 / 0.304
L1b	0.267 / 0.255	0.267 / -	0.338 / 0.156 [†]	0.391 / 0.228	0.381 / 0.117 [†]	0.396 / 0.252	0.350 / 0.278
L1r	0.268 / 0.284	0.260 / -	0.375 / 0.209 [†]	0.383 / 0.268	0.378 / 0.150	0.391 / 0.280	0.384 / 0.299
L2	0.251 / 0.247	0.248 / -	0.372 / 0.209 [†]	0.387 / 0.236	0.377 / 0.133 [†]	0.400 / 0.244	0.378 / 0.294
S1	0.254 / 0.305	0.253 / -	0.346 / 0.196 [†]	0.379 / 0.220	0.358 / 0.099 [†]	0.403 / 0.260	0.371 / 0.284
S2	0.272 / 0.329	0.276 / -	0.386 / 0.182 [†]	0.415 / 0.244	0.394 / 0.098 [†]	0.422 / 0.278	0.402 / 0.315
X1	0.147 / 0.198 [†]	0.160 / -	0.216 / 0.123 [†]	0.207 / 0.156 [†]	0.188 / 0.048 [‡]	0.244 / 0.210	0.231 / 0.243
C_L1b_S1	0.219 / 0.216	0.191 / -	0.263 / 0.136 [†]	0.342 / 0.199	0.337 / 0.082 [†]	0.324 / 0.234	0.279 / 0.228
C_L1r_S1	0.204 / 0.284	0.187 / -	0.279 / 0.206	0.344 / 0.225	0.338 / 0.126 [†]	0.334 / 0.250	0.301 / 0.267
C_S1_L1r	0.224 / 0.262	0.187 / -	0.279 / 0.208	0.326 / 0.241	0.319 / 0.132 [†]	0.327 / 0.254	0.293 / 0.276
C_L2_S4	0.242 / 0.278	0.245 / -	0.334 / 0.182 [†]	0.376 / 0.225	0.376 / 0.117 [†]	0.384 / 0.242	0.351 / 0.287
C_L1r_S3	0.259 / 0.296	0.259 / -	0.350 / 0.160 [†]	0.382 / 0.240	0.376 / 0.145 [†]	0.378 / 0.279	0.365 / 0.296
C_S4_S3	0.270 / 0.334	0.282 / -	0.380 / 0.180 [†]	0.412 / 0.254	0.389 / 0.100 [†]	0.429 / 0.290	0.394 / 0.313
C3_L1r_S3_S4	0.257 / 0.331	0.245 / -	0.333 / 0.180 [†]	0.380 / 0.238	0.375 / 0.135 [†]	0.375 / 0.279	0.348 / 0.276
C3_S1_S3_S4	0.242 / 0.323	0.248 / -	0.316 / 0.181 [†]	0.372 / 0.214	0.358 / 0.062 [†]	0.380 / 0.273	0.342 / 0.289
C3_L1r_S1_S4	0.221 / 0.293	0.200 / -	0.279 / 0.192	0.333 / 0.223	0.330 / 0.111 [†]	0.327 / 0.246	0.287 / 0.262
C4_L1r_S1_S3_S4	0.203 / 0.305	0.205 / -	0.258 / 0.175 [†]	0.325 / 0.216	0.334 / 0.106 [†]	0.331 / 0.253	0.281 / 0.272
C_L1r_X1	0.136 / 0.165 [†]	0.124 / -	0.157 / 0.102 [†]	0.172 / 0.163	0.163 / 0.072 [†]	0.189 / 0.170	0.173 / 0.201
C_X1_S1	0.153 / 0.234	0.169 / -	0.222 / 0.156 [†]	0.203 / 0.161	0.186 / 0.034 [‡]	0.258 / 0.228	0.244 / 0.251
C_S2_X1	0.209 / 0.300	0.217 / -	0.263 / 0.151 [†]	0.252 / 0.150 [†]	0.240 / 0.043 [‡]	0.311 / 0.249	0.272 / 0.277
C_L1r_X1m	0.158 / 0.159 [†]	0.145 / -	0.188 / 0.117 [†]	0.204 / 0.180	0.183 / 0.091 [†]	0.226 / 0.199	0.211 / 0.196
C_S1_X1s	0.206 / 0.282	0.215 / -	0.298 / 0.141 [†]	0.267 / 0.173	0.269 / 0.037 [‡]	0.330 / 0.232	0.293 / 0.268
C3_L1r_S1_X1	0.167 / 0.259	0.147 / -	0.186 / 0.163 [†]	0.195 / 0.156	0.198 / 0.057 [‡]	0.233 / 0.219	0.217 / 0.239
single obfuscation	0.262 / 0.284	0.261 / -	0.363 / -	0.391 / 0.239	0.378 / 0.150 ¹	0.402 / 0.263	0.377 / 0.294
held-out family	0.147 / -	0.160 / -	0.216 / -	0.207 / -	0.188 / -	0.244 / 0.210	0.231 / 0.243
depth 2, all seen	0.236 / 0.278	0.225 / -	0.314 / 0.207 ²	0.364 / 0.231	0.356 / -	0.363 / 0.258	0.330 / 0.278
depth 3, all seen	0.240 / 0.316	0.231 / -	0.309 / 0.192 ¹	0.362 / 0.225	0.354 / -	0.361 / 0.266	0.326 / 0.276
depth 4, all seen	0.203 / 0.305	0.205 / -	0.258 / -	0.325 / 0.216	0.334 / -	0.331 / 0.253	0.281 / 0.272
depth 2, unseen inside	0.172 / 0.272 ³	0.174 / -	0.226 / -	0.220 / 0.169 ⁴	0.208 / -	0.263 / 0.216	0.239 / 0.239
depth 3, unseen inside	0.167 / 0.259	0.147 / -	0.186 / -	0.195 / 0.156	0.198 / -	0.233 / 0.219	0.217 / 0.239

B Reverse-task decomposition

Table 21 splits reverse accuracy four ways for every model and method: execution-graded accuracy, exact recovery of the reference input, format success, and the direction-lock rate of Table 10. It separates reverse reasoning from formatting failure and from answering the forward question instead.

Table 21. **Backward-task decomposition.** For each arm: *exec* execution-graded accuracy, *args* exact-input recovery, *fmt* the fraction of replies meeting the answer contract, and *lock* the direction-lock rate of Table 10. *exec* and *args* are computed on cells clearing the 0.25 format gate; *fmt* and *lock* on all cells. The split separates reverse reasoning from formatting failure and from answering the forward question.

model	Base				ICL				Clean				Breadth				KL				Router				Merge			
	exec	args	fmt	lock	exec	args	fmt	lock	exec	args	fmt	lock	exec	args	fmt	lock	exec	args	fmt	lock	exec	args	fmt	lock	exec	args	fmt	lock
CodeLlama-7B	0.285	0.085	0.94	0.000	0.251	0.077	0.93	0.000	0.278	0.087	0.73	0.081	0.260	0.076	0.93	0.024	0.292	0.091	0.98	0.003	0.290	0.099	0.84	0.015	0.321	0.106	0.96	0.001
CodeLlama-13B	0.290	0.090	0.98	0.000	0.234	0.074	0.91	0.007	0.312	0.114	0.85	0.017	-	-	0.18	0.047	0.224	0.068	0.78	0.085	0.290	0.108	0.72	0.042	0.340	0.126	0.94	0.001
CodeLlama-34B	0.329	0.110	0.97	0.000	0.284	0.093	0.96	0.000	0.294	0.103	0.88	0.004	-	-	0.58	0.027	0.268	0.092	0.86	0.010	-	-	0.55	0.041	0.383	0.140	0.96	0.001
Llama-3.1-8B	0.337	0.089	0.93	0.000	0.342	0.093	0.87	0.000	0.295	0.086	0.92	0.001	0.275	0.076	0.91	0.021	0.284	0.080	0.89	0.024	0.309	0.097	0.97	0.001	0.372	0.109	0.95	0.000
StarCoder2-15B	0.361	0.140	0.58	0.000	0.337	0.122	0.87	0.000	0.348	0.129	0.78	0.152	0.295	0.103	0.71	0.252	0.358	0.134	0.84	0.127	0.368	0.135	0.82	0.133	0.390	0.152	0.91	0.000
Gemma-3-12B	0.389	0.119	0.98	0.000	0.336	0.104	0.89	0.000	0.365	0.127	0.94	0.010	0.399	0.128	0.98	0.002	0.355	0.132	0.62	0.095	0.422	0.141	0.98	0.001	0.420	0.141	0.97	0.000
CodeGemma-7B	0.233	0.064	0.89	0.000	0.222	0.064	0.85	0.000	0.316	0.101	0.97	0.005	0.292	0.094	0.92	0.056	-	-	0.30	0.668	0.297	0.099	0.97	0.008	0.297	0.099	0.97	0.001
Granite-3.1-8B	0.282	0.077	0.79	0.056	0.289	0.082	0.79	0.050	0.207	0.048	0.63	0.181	0.230	0.054	0.90	0.078	0.150	0.031	0.54	0.296	0.252	0.065	0.85	0.091	0.279	0.079	0.96	0.001